



Hausarbeit

Beispieltitel
für einen Bericht

Autor: Vorname Nachname
Matrikel-Nr.: xxxxxxxx
Studiengang: Maschinenbau

Erstprüfer: Prof. Dr. Elmar Wings
Abgabedatum: 21. Juni 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	3
Abbildungsverzeichnis	7
Liste des Listings	9
I. Einleitung	13
1. Einführung	15
2. Aufgabenstellung	17
3. Stand der Technik	19
4. Einbettung des Projektes	21
5. Besondere Herausforderung	23
5.1. Lösung der Herausforderung	23
6. Inhalte der folgenden Kapitel	25
II. Domain Knowledge - Application	27
7. Applikationen	29
III. Domain Knowledge - Hardware	31
8. Arduino Nano 33 BLE Sense	33
8.1. Allgemeine Beschreibung	33
8.2. Mikrocontroller und Leistungsdaten	34
8.3. Betriebsspannung und Logikpegel	34
8.4. Digitale Ein- und Ausgänge	34
8.5. PWM und Servo-Ansteuerung	35
8.6. Zeitmessung und Ablaufsteuerung	35
8.7. USB, Programmierung und serielle Kommunikation	36
8.8. Integrierte Funktionen und Sensorik	36
8.9. Further Readings	37
9. Ultraschallsensor HC-SR04	39
9.1. Allgemeines	39
9.2. Ultraschallsensor HC-SR04	39
9.3. Spezifikation	39
9.4. Bibliothek	40
9.4.1. Beschreibung	40
9.4.2. Installation	40

9.4.3. Funktionen	40
9.5. Beispiel	42
9.5.1. Handbuch	42
9.5.2. Funktion vom Beispiel	43
9.5.3. Code	43
9.6. Kalibrierung	44
9.7. Einfacher Code ohne newPing Bibliothek	44
9.8. Einfache Anwendung	45
9.9. Tests	46
9.9.1. Einfacher Funktionstest	46
9.9.2. Alle Funktionen testen	46
9.10. Weiterführende Literatur	47
10. SG90 Servomotor	49
10.1. Allgemeines	49
10.2. Servomotor TowerPro SG90	49
10.3. Spezifikation	49
10.4. Bibliothek	50
10.4.1. Beschreibung	50
10.4.2. Installation	50
10.4.3. Funktionen	51
10.5. Beispiel	51
10.5.1. Handbuch	51
10.5.2. An einem Beispiel	51
10.5.3. Code	51
10.5.4. Dateien	52
10.6. Kalibrierung	52
10.7. Einfacher Code (ohne Bibliothek)	53
10.8. Einfache Anwendung	54
10.9. Tests	54
10.9.1. Einfacher Funktionstest	54
10.9.2. Alle Funktionen testen	54
10.10. Weiterführende Literatur	54
IV. Domain Knowledge - Tools	55
11. Python Tool XY	57
11.1. Mikrocontroller-Programmierung mit der Arduino IDE	57
11.1.1. Allgemeines zur Arduino IDE	57
11.1.2. Das Steuerungsprogramm des Ultraschallradars	57
11.1.3. Quellcode der Arduino-Steuerung	58
12. Hardware Tool XY	61
12.1. Benutzeroberfläche und Visualisierung mit Processing	61
12.1.1. Allgemeines zu Processing	61
12.1.2. Das Ultraschallradar-Programm	61
12.1.3. Quellcode der Processing-Anwendung	62
V. Development	67
13. Flow Chart Development XY	69

Inhaltsverzeichnis	5
VI. Monitoring	71
14. Monitoring XY	73
VII. Verification - Evaluation - Conclusion	75
15. Evaluation/Verification XY	77
16. To DoX Y	79
17. Conclusion XY	81
VIII. Anhang	83
18. Bill Of Material	85
Stichwortverzeichnis Keywords	87
19. SBOM	89
20. Package Example	91
20.1. Introduction	91
20.2. Description	91
20.3. Installation	91
20.4. Example - Manual	91
20.5. Example	91
20.6. Example - Code	91
20.7. Example - Files	91
20.8. Further Reading	91
IX. Hints - Examples for LaTeX - Read, Use and Remove	93
21. Hinweise	95
21.1. Allgemeine mathematische Beschreibung Bézier-Kurve	96
22. Verrundung mit einem Kreisbogen	99
22.1. Gleichungen	99
22.2. Bewertung	101
22.3. Beispiele	103
23. Maple-Dateien	107
23.1. Geometrien mit Maple	107
23.2. Verwendete Geometrieelemente	107
23.3. Aufbau der Datenstruktur	108
23.4. Struktur eines Moduls	108
23.4.1. Genereller Aufbau eines Moduls	110
23.4.2. Sicherung eines Moduls	112
23.4.3. Verwendung eines Moduls	112
23.4.4. Erstellung eines Maple-Moduls	112
23.5. Funktionen der Module	113
23.5.1. Modul MPoint	113
23.5.2. Modul MLine	114
23.5.3. Modul MArc	115

23.5.4. Modul MBezier	115
23.5.5. Modul MPolygon	116
23.5.6. Modul MGeoList	116
23.5.7. Modul MHermiteProblem	117
23.5.8. Modul MHermiteProblemSym	117
23.5.9. Modul MConstant	118
23.5.10. Modul MGeneralMath	118
23.5.11. Modul Biarc	119
23.5.12. Modul MBiarc	119
23.6. Programmablaufplan	120
23.6.1. Gesamauf	120
23.6.2. Verrundung der Kurve	120
24. Representation of Python Programs	123
25. Representation of Programs written for Arduino Boards	125
26. Erstes Kapitel	127
27. CAGD	129
A. Zeichnungen mit tikz	131
B. Kriterien für eine gutes $\text{\LaTeX}$-Projekt	133
Index	135

Abbildungsverzeichnis

8.1. Arduino Nano 33 BLE Sense – Pinbelegung	33
9.1. Schematische Darstellung des Ultraschallsensors HC-SR04	40
9.2. Schaltplan Ultraschallsensor HC-SR04	41
10.1. Schematische Darstellung des Servomotors TowerPro SG90	50
21.1. Bernstein-Polynom vom Grad 3	97
21.2. Bézier-Kurve zum Beispiel 21.1	98
22.1. Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck	99
22.2. Winkeländerung bei einer Verrundung mit einem Kreisbogen	101
22.3. Krümmungsverlauf bei der Verrundung mit einem Kreisbogen	101
22.4. Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = \text{const}, \alpha)$	102
22.5. Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen	102
23.1. Programmablaufplan „Eckenverrundung“	121
23.2. Programmablaufplan „Auswahl der Verrundungsstrategien“	122

Liste des Listings

9.1. Beispiel Code	43
9.2. Beispiel Code ohne Bibliothekseinbindung	44
9.3. Einparkhilfe Beispielcode	45
10.1. Beispiel Code für den TowerPro SG90	51
10.2. Code ohne Bibilothek für den TowerPro SG90	53
24.1. „Hello World“ in Python – Variant 1	124
25.1. „Hello World“ in Python – Variant 1	126

Abkürzungen

CNC Computerized Numerical Control

SPS Speicherprogrammierbare Steuerung

Teil I.

Einleitung

1. Einführung

Die Umgebungserfassung und die präzise Detektion von Hindernissen sind fundamentale Bestandteile moderner Automatisierungs-, Automobil- und Robotiksysteme. Ein klassisches, aber nach wie vor anschauliches Anwendungsbeispiel hierfür ist das Radar-Prinzip beziehungsweise das hier adaptierte Prinzip des Sonars (Sound Navigation and Ranging).

Im Rahmen dieses Projekts wird ein voll funktionsfähiges Ultraschallradar konzipiert, mechanisch konstruiert und informationstechnisch implementiert. Ziel der Arbeit ist es, das komplexe Zusammenspiel von Mikrocontroller-Hardware, Sensorik, Aktuatorik und einer grafischen Benutzeroberfläche an einem praxisnahen, mechatronischen System zu demonstrieren.

Als zentrale Steuereinheit des Systems dient ein Arduino Nano 33 BLE Sense. Die mechanische Scan-Bewegung wird durch einen 180°-Servomotor vom Typ SG90 realisiert, auf dessen Rotationsachse ein HC-SR-04 Ultraschallsensor montiert ist. Um eine stabile und exakt ausgerichtete Plattform für diese Komponenten zu gewährleisten, wurde im Zuge des Projekts ein maßgeschneidertes Gehäuse im 3D-Druck-Verfahren gefertigt.

Auf der Software-Seite gliedert sich das Projekt in zwei Hauptbereiche: Die hardwarenahe Programmierung des Mikrocontrollers (Embedded C/C++) über die Arduino IDE sowie die Entwicklung einer Benutzeroberfläche am PC. Letzteres wird mithilfe der auf Java basierenden Umgebung Processing umgesetzt. Diese Software empfängt die seriellen Sensordaten des Arduinos (Winkel und Distanz) in Echtzeit und übersetzt sie in eine dynamische, visuell ansprechende Radar-Anzeige, welche erkannte Objekte und Systemzustände unmittelbar visualisiert.

Der vorliegende Bericht dokumentiert den gesamten Entwicklungsprozess dieses Systems. Er erläutert zunächst die verwendeten Hardwarekomponenten, die mechanische Konstruktion sowie den elektronischen Schaltungsaufbau. Anschließend wird detailliert auf die Programmierung und Implementierung der Steuerungs- sowie Visualisierungssoftware eingegangen.

2. Aufgabenstellung

Im Rahmen der Veranstaltung Automatisierungstechnik soll eine Applikation entwickelt werden, die die Leistungsfähigkeit des Arduino Nano 33 BLE Sense Lite sowie ausgewählter Sensoren und Aktoren demonstriert. Die vorliegende Arbeit befasst sich mit der Realisierung eines Systems zur Umgebungserfassung mittels eines Ultraschall-Abstandssensors, der auf einem Servomotor montiert ist und von einem Arduino Nano 33 BLE Sense Lite angesteuert wird. Ziel ist die Umsetzung eines radarartigen Scans der Umgebung und die Visualisierung der gemessenen Distanzen in Form einer geeigneten Darstellung auf dem PC.

Konkret umfasst die Aufgabenstellung die Integration eines Ultraschall-Abstandssensors (HC-SR-04) und eines Servomotors (Tower Pro SG90) in die vorhandene Hardwareplattform. Der Servomotor soll den Sensor in einem definierten Winkelbereich schwenken, während der Mikrocontroller in vorgegebenen Winkelinkrementen Entfernungswerte aufnimmt und diesen Messdaten einen eindeutigen Winkel zuordnet. Auf Basis dieser Messwerte ist eine Auswertung zu konzipieren, die eine übersichtliche Darstellung der Umgebung ermöglicht.

Neben der reinen Funktionserfüllung werden weitere Anforderungen an die Applikation gestellt. Dazu gehören eine strukturierte und modular aufgebaute Firmware, die Verwendung geeigneter Bibliotheken zur Ansteuerung von Sensor und Aktor. Außerdem sind Verfahren zur Visualisierung der Live-Daten auf einem PC zu entwickeln und zu implementieren, sodass die resultierende Umgebungskarte in Echtzeit nachvollzogen werden kann.

Die Aufgabenstellung umfasst weiterhin die Erstellung einer Entwickler- und Service-Dokumentation. Diese beinhaltet neben der Projektbeschreibung eine detaillierte Beschreibung der Hardware des Arduino Nano 33 BLE Sense Lite und der eingesetzten Komponenten, die Spezifikation und Durchführung von Tests, eine Darstellung der Softwarearchitektur sowie Angaben zu Bezugsquellen und Stückliste (Bill of Materials).

3. Stand der Technik

Ultraschall-Abstandssensoren werden seit vielen Jahren in der Automatisierungstechnik, Robotik und Fahrzeugtechnik zur berührungslosen Distanzmessung eingesetzt.[Leu] Sie arbeiten in der Regel nach dem Laufzeitmessprinzip, bei dem ein kurzer Ultraschallimpuls ausgesendet und die Zeit bis zum Empfang des reflektierten Echos gemessen wird. Aus der Schalllaufzeit lässt sich unter Annahme einer bekannten Schallgeschwindigkeit der Abstand zu einem Hindernis bestimmen. Im Niedrigkostenbereich haben sich dafür Sensormodule wie der HC-SR-04 etabliert, die eine einfache Anbindung an Mikrocontroller über Trigger- und Echo-Signale erlauben.

Für die mechanische Ausrichtung von Sensoren und leichten Lasten werden in zahlreichen Anwendungen kompakte RCServomotoren eingesetzt. Diese enthalten neben einem Gleichstrommotor ein Getriebe sowie eine integrierte Positionsregelung, die über ein Pulsweitenmodulationssignal (PWM) angesteuert wird. Miniaturservos wie der Tower Pro SG90 sind vor allem in ferngesteuerten Modellen wie Flugzeugen und Helikoptern verbreitet, wo sie etwa Ruderflächen oder andere bewegliche Komponenten präzise ansteuern.[kts:2026] In technischen Lehr- und Demonstrationsaufbauten werden derartige Servos zudem häufig genutzt, um Sensoren in einem begrenzten Winkelbereich gezielt zu positionieren.

Die Kombination aus Ultraschall-Abstandssensor und Servomotor ermöglicht die Realisierung kostengünstiger Systeme zur zweidimensionalen Umgebungserfassung. Durch das sequenzielle Abtasten verschiedener Winkelpositionen kann ein Scan der Umgebung erzeugt werden. In frei verfügbaren Projekten und Tutorials wird dieses Prinzip typischerweise mithilfe eines Ultraschallmoduls des Typs HC-SR-04, eines SG90-Servos und eines Arduino Uno oder Arduino Nano umgesetzt. (vgl. [pro15]) Die gemessenen Distanzen werden dabei entweder auf einem seriell angebundenen PC visualisiert oder direkt auf einem Display dargestellt. Derartige Beispiele dienen häufig als Einführung in die Sensordatenerfassung und -visualisierung mit einfachen Mikrocontrollersystemen.

4. Einbettung des Projektes

Das in diesem Projekt realisierte System knüpft an den beschriebenen Stand der Technik an, indem es die etablierte Kombination aus Ultraschall-Abstandssensor HC-04 und Tower Pro SG90-Servomotor zur Umgebungserfassung verwendet. Der Ultraschallsensor ist auf dem Servomotor montiert und wird in einem definierten Winkelbereich geschwenkt, sodass in diskreten Winkelpositionen Entfernungswerte aufgenommen und zu einem Umgebungsbild zusammengeführt werden.

Viele veröffentlichte Beispielprojekte setzen noch auf Arduino-Boards mit 8Bit-Mikrocontrollern.[Rit21] In der vorliegenden Applikation kommt dagegen der Arduino Nano 33 BLE Sense als Steuerplattform zum Einsatz. Diese Hardwarebasis ermöglicht eine strukturierte Implementierung der Mess- und Auswertalgorithmen sowie die Erweiterung des klassischen Ultraschall-Radar-Konzepts um weitergehende Funktionen, etwa eine flexible Visualisierung der Messdaten auf einem PC.

Durch diese Ausgestaltung ordnet sich das Projekt in die Gruppe kostengünstiger Ultraschall-Radar-Systeme ein, entwickelt diese jedoch durch den Einsatz moderner Hardware und eine systematische Softwarestruktur weiter.[Rit21] Es dient damit sowohl als Demonstrator für die Sensordatenerfassung mit dem Arduino Nano 33 BLE Sense als auch als Grundlage für weiterführende Anwendungen in der Automatisierungstechnik.

5. Besondere Herausforderung

Im Rahmen der Entwicklung der Applikation traten mehrere besondere Herausforderungen auf, die sowohl die mechanische Konstruktion als auch die Softwaregestaltung betrafen. Ein zentrales Ziel war der möglichst kompakte Aufbau des Systems. Dabei musste das Gehäuse so dimensioniert und gestaltet werden, dass alle Komponenten, insbesondere der Ultraschallsensor, der Servomotor und das ArduinoBoard, platzsparend integriert werden konnten, ohne die Funktionalität oder Zugänglichkeit zu beeinträchtigen. Eine zusätzliche Herausforderung bestand in der geordneten Führung und Platzierung der Anschlusskabel im Inneren des Gehäuses, um mechanische Kollisionen, Kabelbrüche sowie Beeinflussungen der Sensorik durch bewegte Leitungen zu vermeiden.

Auf der Softwareseite ergaben sich weitere Anforderungen aus der gewünschten Plattformunabhängigkeit der Benutzerschnittstelle. Es sollte eine Anwendung bereitgestellt werden, die sich direkt von einem USB Stick starten lässt und möglichst ohne zusätzliche Installation auf unterschiedlichen Betriebssystemen lauffähig ist. Dies erforderte eine sorgfältige Auswahl der eingesetzten Technologien und Bibliotheken, um Abhängigkeiten zu reduzieren und eine robuste Ausführung zu ermöglichen.

Schließlich zeigte sich im Verlauf der Entwicklung, dass die Visualisierung der Messdaten auf der Benutzeroberfläche zunächst zu überdimensioniert und unübersichtlich war. Einzelne Objekte wurden zu groß dargestellt, sodass die eigentlichen strukturellen Informationen der Umgebung nur eingeschränkt erkennbar waren.

5.1. Lösung der Herausforderung

6. Inhalte der folgenden Kapitel

Teil II.

Domain Knowledge - Application

7. Applikationen

Teil III.

Domain Knowledge - Hardware

8. Arduino Nano 33 BLE Sense

8.1. Allgemeine Beschreibung

Der Arduino Nano 33 BLE Sense ist ein kompaktes Mikrocontrollerboard aus der Arduino-Nano-Serie. Es eignet sich für eingebettete Anwendungen, bei denen Sensoren ausgelesen, Aktoren angesteuert und Messdaten verarbeitet werden müssen. Durch seine kleine Bauform kann das Board gut in Versuchsaufbauten, Prototypen und mobile Systeme integriert werden.

Wie andere Arduino-Boards dient der Nano 33 BLE Sense als programmierbare Steuereinheit. Er kann digitale und analoge Signale verarbeiten, zeitabhängige Abläufe steuern und über Schnittstellen mit externen Bauteilen kommunizieren. Dadurch kann er in vielen mechatronischen Anwendungen als Bindeglied zwischen Software, Sensorik und Aktorik eingesetzt werden.

Obwohl der Arduino Nano 33 BLE Sense mehrere integrierte Sensoren besitzt, muss ein Projekt diese nicht zwingend verwenden. Das Board kann auch ausschließlich zur Ansteuerung externer Komponenten eingesetzt werden. In diesem Fall stehen vor allem die Rechenleistung des Mikrocontrollers, die digitalen Ein- und Ausgänge, die Zeitmessung, die PWM-Ausgabe und die serielle Kommunikation im Vordergrund.

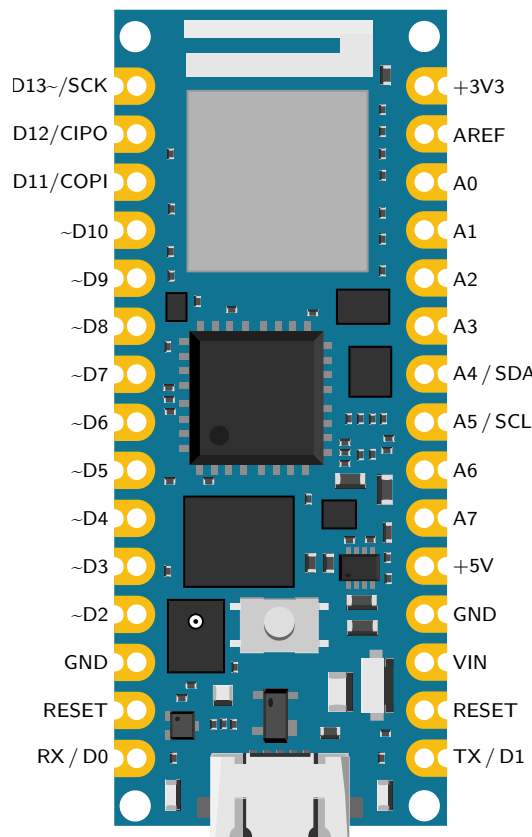


Abbildung 8.1.: Arduino Nano 33 BLE Sense – Pinbelegung

8.2. Mikrocontroller und Leistungsdaten

Der Arduino Nano 33 BLE Sense basiert auf dem Nordic nRF52840 Mikrocontroller. Dieser enthält einen Arm Cortex-M4F-Prozessor mit einer Taktfrequenz von 64 MHz. Laut Arduino-Datenblatt besitzt das Board 1 MB Flash-Speicher und 256 KB SRAM [Ard26a]. Damit verfügt das Board über ausreichend Leistung für typische Steuerungs-, Mess- und Auswertungsaufgaben.

Der Flash-Speicher dient zur Speicherung des Programmcodes. Der SRAM wird während der Programmausführung für Variablen, Messwerte und Zwischenergebnisse verwendet. Für Anwendungen wie das Einlesen eines Sensors, das Erzeugen eines Steuersignals oder das Berechnen einfacher physikalischer Grössen ist die vorhandene Rechenleistung in der Regel ausreichend.

Eigenschaft	Wert/ Bedeutung
Mikrocontroller	Nordic nRF52840
Prozessorkern	Arm Cortex-M4F
Taktfrequenz	64 MHz
Flash-Speicher	1 MB für Programmcodes
SRAM	256 KB für Variablen und Zwischenspeicher
Betriebsspannung	3,3 V Logikspannung
Digitale Ein- und Ausgänge	Anschluss externer Sensoren und Aktoren
USB	Programmierung und serielle Kommunikation

Tabelle 8.1.: Wichtige Kenndaten des Arduino Nano 33 BLE Sense

8.3. Betriebsspannung und Logikpegel

Ein wichtiger Punkt bei der Verwendung des Arduino Nano 33 BLE Sense ist die Betriebsspannung der Ein- und Ausgänge. Das Board arbeitet mit einer Logikspannung von 3,3 V und ist laut Arduino-Datenblatt nicht 5-V-tolerant [Ard26a]. Das bedeutet, dass an den digitalen und analogen Eingangspins maximal 3,3 V anliegen dürfen. Höhere Spannungen, wie sie beispielsweise bei vielen 5-V-Sensoren oder älteren Arduino-Modulen auftreten, können den Mikrocontroller beschädigen[Bri].

Dies ist insbesondere bei der Kombination mit externen Sensoren und Aktoren relevant. Viele elektronische Module werden mit 5 V betrieben und geben ihre Ausgangssignale ebenfalls mit einem Pegel von 5 V aus. Vor dem Anschluss solcher Komponenten muss daher geprüft werden, ob deren Signalpegel mit dem Arduino Nano 33 BLE Sense kompatibel sind.

Falls ein Ausgangssignal eines externen Moduls oberhalb von 3,3 V liegt, ist eine Pegelanpassung erforderlich. Diese kann beispielsweise mit einem Spannungsteiler oder einem Pegelwandler umgesetzt werden. Ein Spannungsteiler reduziert die Eingangsspannung mithilfe zweier Widerstände auf einen für den Mikrocontroller zulässigen Wert. Ein Pegelwandler eignet sich insbesondere dann, wenn Signale bidirektional übertragen werden oder eine robustere Anpassung zwischen unterschiedlichen Logikspannungen erforderlich ist[Jim].

Die Berücksichtigung der Logikpegel ist somit sowohl für die Funktionsfähigkeit der Schaltung als auch für den Schutz des Mikrocontrollers von Bedeutung.

8.4. Digitale Ein- und Ausgänge

Die digitalen Anschlüsse des Arduino Nano 33 BLE Sense können je nach Anwendung als Eingang oder Ausgang verwendet werden. Wird ein Pin als Ausgang konfiguriert, kann der Mikrocontroller darüber ein digitales Signal ausgeben, das entweder

den Zustand High oder Low besitzt. Bei einer Konfiguration als Eingang kann der Mikrocontroller den anliegenden Signalzustand erfassen und auswerten.[Arda]

Auf diese Weise lassen sich verschiedene externe Komponenten mit dem Arduino verbinden. Digitale Eingänge können beispielsweise zum Einlesen von Tastern oder Sensorsignalen genutzt werden. Digitale Ausgänge eignen sich hingegen zur Ansteuerung von LEDs, Treiberschaltungen oder anderen elektronischen Baugruppen[Arda]. Neben dem einfachen Erfassen von High- und Low-Zuständen kann der Mikrocontroller auch zeitabhängige digitale Signale auswerten. Dabei wird nicht nur erkannt, ob ein Signal anliegt, sondern auch, wie lange ein bestimmter Signalzustand andauert. Diese Funktion ist besonders bei Sensoren relevant, die ihre Messinformation über die Dauer eines Impulses ausgeben[Arda].

Ein Beispiel dafür ist der Ultraschallsensor HC-SR04. Bei diesem Sensor wird die Messung durch einen kurzen Trigger-Impuls gestartet. Laut Datenblatt reicht dafür ein Impuls mit einer Dauer von 10 μ s aus. Anschließend gibt der Sensor am Echo-Ausgang ein Signal aus, dessen Impulsdauer von der Entfernung zum reflektierenden Objekt abhängt.

8.5. PWM und Servo-Ansteuerung

Neben der Verarbeitung einfacher digitaler Signale kann der Arduino Nano 33 BLE Sense auch pulswidenmodulierte Signale ausgeben. Bei der Pulsweitenmodulation wird ein digitales Signal in regelmäßigen Zeitabständen ein- und ausgeschaltet. Dabei ist nicht nur die Periodendauer des Signals entscheidend, sondern vor allem die Dauer des High-Zustands innerhalb einer Periode. Dieses Verhältnis wird als Tastgrad bezeichnet und kann zur Ansteuerung verschiedener elektronischer Komponenten genutzt werden[Ardc].

Ein typisches Anwendungsbeispiel für pulswidenmodulierte Signale ist die Ansteuerung von Servomotoren. Der Tower Pro SG90 ist ein kompakter Modellbau-Servomotor, dessen Winkelposition über eine Signalleitung vorgegeben wird. Dazu erhält der Servo ein periodisches Steuersignal mit einer Periodendauer von etwa 20 ms. Die gewünschte Position wird über die Breite des High-Impulses bestimmt. Ein kurzer Impuls von ungefähr 1 ms führt zu einer Endposition, ein Impuls von etwa 1,5 ms zur Mittelstellung und ein Impuls von ungefähr 2 ms zur gegenüberliegenden Endposition.

Der Arduino kann diese Steuersignale über einen geeigneten digitalen Ausgang erzeugen und den Servomotor dadurch auf unterschiedliche Winkelpositionen einstellen. Dadurch lassen sich einfache Bewegungsabläufe realisieren, beispielsweise das Schwenken eines Sensors oder das Positionieren kleiner mechanischer Bauteile. Da ein Servomotor beim Anlaufen oder unter Last deutlich mehr Strom benötigt als ein Mikrocontroller-Pin bereitstellen kann, darf die Versorgung des Motors nicht direkt über einen digitalen Ausgang erfolgen. Der Ausgangspin dient nur zur Signalübertragung, während die Stromversorgung des Servos separat ausreichend dimensioniert werden muss[Ardd].

8.6. Zeitmessung und Ablaufsteuerung

Eine wichtige Aufgabe von Mikrocontrollern ist die zeitliche Steuerung von Abläufen. Der Arduino kann Signale zu definierten Zeitpunkten ausgeben, kurze Impulse erzeugen und die Dauer externer Signale messen. Dadurch lassen sich Sensorabfragen, Bewegungsabläufe und Berechnungen in einer festgelegten Reihenfolge ausführen.

Bei Anwendungen mit Sensoren und Aktoren ist diese zeitliche Reihenfolge besonders wichtig. Wird beispielsweise ein Sensor mechanisch bewegt, sollte die Messung erst erfolgen, nachdem die gewünschte Position erreicht wurde. Andernfalls können die Messwerte ungenau oder schwer interpretierbar sein. Ebenso benötigen manche Sensoren einen Mindestabstand zwischen zwei Messungen, damit sich aufeinanderfolgende

Messvorgänge nicht gegenseitig beeinflussen.

Ein allgemeiner Ablauf für eine kombinierte Sensor-Aktor-Anwendung kann daher aus mehreren Schritten bestehen:

1. Aktor auf eine gewünschte Position stellen.
2. Kurze Wartezeit einhalten, bis die Bewegung abgeschlossen ist.
3. Sensorabfrage auslösen.
4. Eingangssignal messen.
5. Messwert berechnen oder auswerten.
6. Ergebnis speichern oder über eine Schnittstelle ausgeben.

8.7. USB, Programmierung und serielle Kommunikation

Der Arduino Nano 33 BLE Sense wird über eine USB-Verbindung mit dem Computer verbunden. Diese Verbindung dient einerseits zur Spannungsversorgung des Boards und andererseits zur Übertragung des Programms auf den Mikrocontroller. In der Arduino IDE wird dazu das passende Board sowie der zugehörige Anschluss ausgewählt. Anschließend kann der Quellcode kompiliert und auf das Board geladen werden [Ardb]. Neben der Programmübertragung kann die USB-Verbindung auch für die serielle Kommunikation zwischen Mikrocontroller und Computer genutzt werden. Dafür stellt Arduino die serielle Schnittstelle zur Verfügung. Über Befehle wie `Serial.begin()` und `Serial.println()` können Daten vom Mikrocontroller an den Computer gesendet und im seriellen Monitor angezeigt werden [Ardb].

Die serielle Ausgabe ist insbesondere während der Entwicklung und Inbetriebnahme eines Versuchsaufbaus hilfreich. Über den seriellen Monitor können beispielsweise Rohdaten, berechnete Messwerte oder Statusmeldungen ausgegeben werden. Dadurch lässt sich überprüfen, ob angeschlossene Sensoren plausible Werte liefern, ob Steuersignale korrekt erzeugt werden und ob die Verarbeitung der Messdaten im Programm wie vorgesehen abläuft.

8.8. Integrierte Funktionen und Sensorik

Der Arduino Nano 33 BLE Sense verfügt neben den allgemeinen Mikrocontrollerfunktionen über mehrere integrierte Sensoren sowie eine Bluetooth-Low-Energy-Schnittstelle. Zur Sensorik des Boards gehören eine Inertialmesseinheit zur Erfassung von Beschleunigung, Winkelgeschwindigkeit und Magnetfeld, ein Umgebungslicht-, Farb-, Gesten- und Näherungssensor, ein barometrischer Drucksensor, ein Temperatur- und Feuchtigkeitssensor sowie ein digitales Mikrofon [Ard26a].

Diese integrierten Komponenten erweitern die Einsatzmöglichkeiten des Boards deutlich. Sie ermöglichen beispielsweise Anwendungen in den Bereichen Bewegungserkennung, Umgebungsüberwachung, Gestensteuerung, Audioverarbeitung und drahtlose Datenübertragung. Welche Sensoren oder Schnittstellen tatsächlich genutzt werden, hängt jedoch vom jeweiligen Aufbau und der Zielsetzung der Anwendung ab.

Im Rahmen dieses Projekts werden die Onboard-Sensoren des Arduino Nano 33 BLE Sense nicht verwendet. Stattdessen dient das Board als zentrale Steuereinheit für externe Komponenten. Dazu zählen der Ultraschallsensor HC-SR04 zur Entfernungsmessung und der Servomotor Tower Pro SG90 zur Ausrichtung des Sensors. Im Vordergrund stehen daher nicht die integrierte Sensorik des Boards, sondern die digitalen Ein- und Ausgänge, die Zeitmessung, die Signalverarbeitung sowie die Ansteuerung des Servomotors.

8.9. Further Readings

9. Ultraschallsensor HC-SR04

Einleitung

Dieses Kapitel befasst sich mit der berührungslosen Abstandsmessung mithilfe von Sensoren. Zunächst wird ein allgemeiner Überblick über Ultraschallsensoren zur Distanzmessung gegeben. Anschließend wird der in diesem Projekt verwendete Ultraschallsensor HC-SR04 detailliert beschrieben, einschließlich seiner technischen Spezifikationen und der softwareseitigen Implementierung unter Verwendung ausgewählter C++ Bibliotheken.

9.1. Allgemeines

Sensoren zur Abstandsmessung sind wichtige Komponenten in der modernen Automatisierungstechnik, Robotik und Fahrzeugtechnik [Fra16]. Sie ermöglichen es Systemen, eine Umgebung räumlich zu erfassen. Ultraschallsensoren arbeiten nach dem Echo-Lot-Prinzip (Schalllaufzeitmessung). Sie senden Schallwellen aus und messen die Zeit, bis das Echo zurückkehrt [Ele20]. Sie sind unabhängig von der Farbe oder optischen Transparenz eines Objekts, haben jedoch ihre Grenze bei schallabsorbierenden Materialien (z. B. Schaumstoff) [Fra16].

9.2. Ultraschallsensor HC-SR04

In diesem Projekt kommt der Ultraschallsensor **HC-SR04** zum Einsatz. Er wandelt die gemessene Entfernung in ein PWM-Signal um, welches am Ausgang zur Verfügung steht [KT-12]. Das Auslösen eines Messzyklus geschieht durch eine fallende Flanke am TRIG-Eingang für mindestens 10 μ s. Das Modul sendet darauf nach ca. 250 μ s ein 40 kHz Burst-Signal für die Dauer von 200 μ s aus. Danach geht der ECHO-Ausgang sofort auf HIGH-Pegel und das Modul wartet auf den Empfang des Echos. Wird dieses detektiert, fällt der Ausgang auf LOW-Pegel. Wird kein Echo detektiert, verweilt der Ausgang für insgesamt 200 ms auf HIGH-Pegel und zeigt so eine erfolglose Messung an. Ein vollständiges Messintervall hat eine Dauer von 20 ms, sodass maximal 50 Messungen pro Sekunde durchgeführt werden können [KT-12].

9.3. Spezifikation

Die elektrischen und physikalischen Parameter entstammen den offiziellen Spezifikationen von KT-elektronic [KT-12].

- Betriebsspannung (VCC: +5 V DC $\pm 10\%$)
- Stromaufnahme: < 2 mA
- Messbare Distanz: 2 cm bis ca. 300 cm
- Messauflösung: 3 mm
- Messintervall: 20 ms (maximal 50 Messungen pro Sekunde)

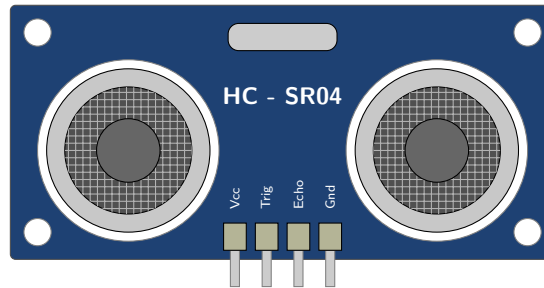


Abbildung 9.1.: Schematische Darstellung des Ultraschallsensors HC-SR04

- Messwinkel: Bis zu 45° bei kurzen Distanzen (< 1 m), Sendekegel von 15° bei maximaler Distanz
- Abmessungen (L x B x T): 45 x 21 x 18 mm

Schaltplan Das Modul besitzt vier Pins (siehe Abbildung 9.1). Der VCC-Pin ist mit der 5V-Spannungsversorgung des Mikrocontrollers verbunden, der GND-Pin (Pin 4) mit dem GND-Pin des Mikrocontrollers. Der TRIG-Eingang (Pin 2) und der ECHO-Ausgang (Pin 3) arbeiten mit TTL-Pegel (siehe Abbildung 9.2). Hinweis bei 3.3V-Mikrocontrollern: Da der ECHO-Pin ein 5V-Signal liefert, muss dieser zwingend durch einen Spannungsteiler an den 3.3V-Eingang des Mikrocontrollers angepasst werden [KT-12].

9.4. Bibliothek

9.4.1. Beschreibung

Für die Ansteuerung des Sensors wird die externe C++ Bibliothek **NewPing** von Tim Eckel verwendet [Eck23].

9.4.2. Installation

Die Bibliothek kann direkt über den internen Bibliotheksverwalter der Arduino IDE installiert werden [Eck23].

- **Data types:** Messzeiten in Mikrosekunden und Distanzen in Zentimetern werden als `unsigned int` deklariert, um Speicherplatz zu sparen und negative Distanzen auszuschließen.
- **Data structure:** Der Sensor wird als Objekt der Klasse `NewPing` instanziiert.
- **Data quantity:** Das Softwaredesign ist ressourcenschonend konzipiert. Es können bis zu 15 Ultraschallsensoren gleichzeitig und asynchron über ein einziges Board betrieben werden, bei einer Abtastrate von bis zu 30 Pings pro Sekunde je Sensor.
- **Data quality:** Die Bibliothek bietet integrierte digitale Filter. Die Funktion `pingmedian` verarbeitet mehrere Messwerte zu einem Median, was Rauschen und fehlerhafte Echos effektiv aus dem Datensatz entfernt.

9.4.3. Funktionen

Die wichtigsten Befehle der NewPing-Bibliothek umfassen [Eck23]:

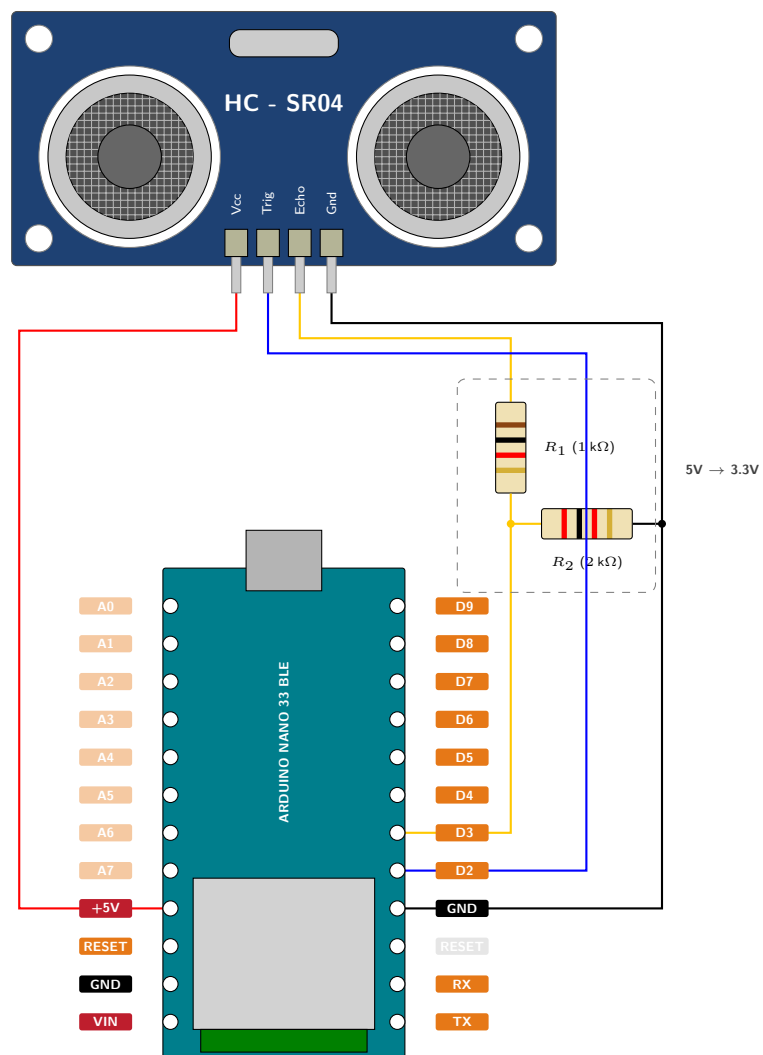


Abbildung 9.2.: Schaltplan Ultraschallsensor HC-SR04

- `NewPing sonar`(`TRIGGER_PIN`, `ECHO_PIN`, `MAX_DISTANCE` 9.1): Konstruktor zur Erstellung des Sensor-Objekts.
 - **TRIGGER_PIN (Der Auslöser):** Dieser Anschluss entspricht Pin 2 auf dem HC-SR04 Modul und arbeitet mit einem regulären TTL-Pegel [KT-12]. Um einen neuen Messzyklus auszulösen, muss an diesem Triggereingang ein Signal mit einer fallenden Flanke für mindestens 10 μ s anliegen [KT-12]. Als direkte Reaktion auf dieses Signal sendet das Modul selbstständig ein 40 kHz Burst-Signal aus [KT-12]. In dem vorliegenden Code-Beispiel wird dieser Anschluss dem digitalen Pin 12 des Arduino zugewiesen.
 - **ECHO_PIN (Der Empfänger):** Dieser Ausgang liegt auf Pin 3 des Sensormoduls und gibt das eigentliche Messergebnis ebenfalls als TTL-Pegel aus [KT-12]. Unmittelbar nach dem Aussenden des Ultraschall-Bursts springt dieser Pin auf einen HIGH-Pegel und wartet auf den Empfang des zurückkehrenden Echos [KT-12]. Sobald das reflektierte Signal detektiert wird, fällt der Ausgang sofort auf einen LOW-Pegel ab [KT-12]. Wird gar kein Echo erkannt (weil der Raum leer ist), verweilt der Pin für exakt 200 ms auf dem HIGH-Pegel, um dem Mikrocontroller die erfolglose Messung zu signalisieren [KT-12]. In der Software ist dieser Pin dem digitalen Pin 11 zugewiesen.
 - **MAX_DISTANCE (Die maximale Reichweite):** Rein physikalisch besitzt der HC-SR04 eine messbare Distanz von 2 cm bis zu ca. 300 cm [KT-12]. Soll die maximale Distanz von 3 Metern erreicht werden, muss der Sensor sehr exakt auf das Objekt zielen, und es dürfen keine störenden Gegenstände im Sendekegel von 15° vorhanden sein [KT-12]. Im Programmcode ist `MAX_DISTANCE` eine festgelegte Konstante (hier auf 200 cm gesetzt), die dem `NewPing`-Objekt mitteilt, bis zu welcher Entfernung der Sensor überhaupt auf Objekte reagieren soll [Eck23].
- `sonar.ping()`: Sendet einen Ping und gibt die gemessene Laufzeit in Mikrosekunden (μ s) zurück.
- `sonar.ping_cm()`: Führt eine Messung durch und gibt die in Zentimeter (cm) umgerechnete Distanz zurück.
- `sonar.ping_median(int iterations)`: Führt die angegebene Anzahl an Pings durch, verwirft Ausreißer und gibt die mediane Laufzeit zurück.

9.5. Beispiel

In diesem Abschnitt wird ein einfaches, eigenständiges Codebeispiel beschrieben, welches die Funktionsweise und Integration der Bibliothek in C++ verdeutlicht.

9.5.1. Handbuch

1. Verbinden Sie VCC und GND des HC-SR04 mit 5V und GND des Boards [KT-12].
2. Verbinden Sie den TRIG-Pin mit Pin 12 und den ECHO-Pin mit Pin 11.
3. Öffnen Sie die Arduino IDE, kompilieren Sie den Code und laden Sie ihn hoch.
4. Öffnen Sie den Seriellen Monitor (Baudrate 115200). Hier werden nun die Messergebnisse ausgegeben[Ele20].

9.5.2. Funktion vom Beispiel

Das Skript bindet die `NewPing.h` ein. Über `#define` werden Pins und der maximale Messbereich (200 cm) definiert. In der Hauptschleife (`loop()`) stellt `delay(50)` sicher, dass zwischen zwei Pings gewartet wird, um Echosüberlappungen zu verhindern [Eck23]. Anschließend wird `ping_median(5)` aufgerufen. Ist ein Objekt vorhanden, muss die gemessene Zeit in eine Längeneinheit umgewandelt werden. Hierfür kommt der bibliotheksinterne Befehl `US_ROUNDTRIP_CM` zum Einsatz. Dies ist ein vordefinierter Teilerfaktor, mit dem Wert 57. Er basiert auf der physikalischen Schallgeschwindigkeit in der Luft. Da der Schall für einen Zentimeter Wegstrecke etwa 29 Mikrosekunden benötigt und das Signal den Weg doppelt zurücklegen muss (Hin- und Rückweg), entspricht 1 cm Objektstand exakt einer Signallaufzeit von ca. 57 bis 58 Mikrosekunden. Durch die Division `uS / US_ROUNDTRIP_CM` wird die gemessene Zeit somit effizient und direkt in den korrekten Zentimeter-Abstand umgerechnet [Eck23].

9.5.3. Code

Listing 9.1.: Beispiel Code

```

1000 \begin{verbatim}
// File: Beispielcode.ino
1002 // Standalone script for filtered distance measurement with HC-SR04.
// Uses NewPing library to get a median value from multiple measurements
// to reduce noise.
1004 // Author: Wilko Hinrichs
// Date: 2026-04-27
1006
#include <NewPing.h>
1008
// Digital pin for trigger signal
1010 #define TRIGGER_PIN 12
1012
// Digital pin for echo signal
#define ECHO_PIN 11
1014
// Maximum distance for the sensor (in cm)
1016 #define MAX_DISTANCE 200
1018
// Initialize NewPing object
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
1020
void setup() {
1022   Serial.begin(115200);
   Serial.println("System gestartet. Beginne Messung...");
1024 }

1026 void loop() {
// Wait between pings (min. 29ms recommended)
1028   delay(50);

1030   // Get median of 5 measurements (in microseconds)
   unsigned int uS = sonar.ping_median(5);
1032
   Serial.print("Gefilterte Distanz: ");
1034
   if (uS == 0) {
1036     Serial.println("Außerhalb der Reichweite");
   } else {
1038     // Convert microseconds to centimeters
     Serial.print(uS / US_ROUNDTRIP_CM);
1040     Serial.println(" cm");
   }
1042 }
\end{verbatim}

```

../Code/Ultraschallsensor/Beispielcode/Beispielcode.ino

9.6. Kalibrierung

Die Ultraschallmessung beruht auf der Ausbreitungsgeschwindigkeit von Schall in der Luft. Um das System zu überprüfen wird ein Objekt in einem exakt gemessenem Abstand vor dem Sensor aufgestellt und mit der Ausgabe im seriellen Monitor verglichen. Die Standardberechnung der Bibliothek geht von einer konstanten Schallgeschwindigkeit von $c \approx 343 \text{ m/s}$ bei 20°C aus [Fra16]. Da die Schallgeschwindigkeit temperaturabhängig ist, ändert sie sich um ca. $0,6 \text{ m/s}$ pro Grad Celsius [Fra16]. Für hochpräzise Anwendungen muss das System kalibriert werden (z.B. über einen DHT22-Sensor). Die dynamische Berechnung lautet:

$$\text{Distanz} = \frac{\text{Laufzeit}}{2} \times (331,3 + 0,606 \times \text{Temperatur in } ^\circ\text{C})$$

9.7. Einfacher Code ohne newPing Bibliothek

Der folgende Code demonstriert eine Abstandsmessung ohne Einbindung der NewPing-Bibliothek und nutzt `pulseIn()`.

Listing 9.2.: Beispiel Code ohne Bibliothekseinbindung

```

1000 \begin{verbatim}
// File: Beispielcode2.ino
1002 // Standalone script for filtered distance measurement with HC-SR04.
// Uses NewPing library to get a median value from multiple measurements
// to reduce noise.
1004 // Author: Wilko Hinrichs
// Date: 2026-04-27
1006
const int TRIG_PIN = 12; // Pin for trigger signal
1008 const int ECHO_PIN = 11; // Pin for echo signal
1010
void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  Serial.begin(115200);
1014 }
1016
void loop() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
1018
  // 10 microseconds HIGH pulse (ends with falling edge)
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
1024
  // pulseIn measures the length of the ECHO pulse
  long duration = pulseIn(ECHO_PIN, HIGH);
1026
  // Calculate distance (travel time in us / 58)
  float distance = duration / 58.0;
1030
  Serial.print("Gemessene Distanz: ");
  Serial.print(distance);
  Serial.println(" cm");
1032

```

```

1034 // Keep interval (> 20ms)
1036 delay(50);
    }
1038 \end{verbatim}

```

../Code/Ultraschallsensor/Beispielcode2/Beispielcode2.ino

9.8. Einfache Anwendung

Eine anschauliche Anwendung für den HC-SR04 ist eine automatisierte Einparkhilfe. Fährt ein Fahrzeug heran, liest der Sensor die Distanz. Unterschreitet der distance-Wert 50 cm, schaltet das System eine Warn-LED ein. Fällt der Wert auf unter 10 cm, wird ein Buzzer getriggert [Eck23].

Listing 9.3.: Einparkhilfe Beispielcode

```

1000 \begin{verbatim}
    // File: EinparkhilfeCode.ino
1002 // Application example: Automated parking assistant.
    // System shows distance via LED and warns acoustically on critical
    // approach.
1004 // Author: Wilko Hinrichs
    // Date: 2026-04-28
1006
    #include <NewPing.h>
1008
    // Pin configuration
1010 #define TRIGGER_PIN 12
    #define ECHO_PIN 11
1012 #define MAX_DISTANCE 200 // Maximum scan range in cm
    #define LED_PIN 3 // Yellow warning LED
1014 #define BUZZER_PIN 4 // Acoustic buzzer
1016
    // Initialize sensor
    NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
1018
    void setup() {
1020     Serial.begin(115200);
        pinMode(LED_PIN, OUTPUT);
1022     pinMode(BUZZER_PIN, OUTPUT);
    }
1024
    void loop() {
1026     // Keep measurement interval
        delay(50);
1028
        // Measure distance in cm
1030     unsigned int distance = sonar.ping_cm();
1032
        Serial.print("Abstand: ");
        Serial.print(distance);
1034     Serial.println(" cm");
1036
        // Logic check based on distance thresholds
        if (distance > 0 && distance < 10) {
1038             // Critical area: LED and buzzer active
            digitalWrite(LED_PIN, HIGH);
1040             digitalWrite(BUZZER_PIN, HIGH);
        }
1042     else if (distance >= 10 && distance < 50) {
        // Warning area: Only LED active
1044         digitalWrite(LED_PIN, HIGH);
            digitalWrite(BUZZER_PIN, LOW);
    }
    }
\end{verbatim}

```

```

1046 }
1048 else {
1049     // Safe area: All off
1050     digitalWrite(LED_PIN, LOW);
1051     digitalWrite(BUZZER_PIN, LOW);
1052 }
\end{verbatim}

```

../Code/Ultraschallsensor/EinparkhilfeCode/EinparkhilfeCode.ino

9.9. Tests

9.9.1. Einfacher Funktionstest

Für die Überprüfung der grundlegenden Schaltung und Sensorik wird das Standalone-Skript aus Abschnitt 9.1 (Gefilterte Distanzmessung mit der NewPing-Bibliothek) auf den Mikrocontroller geladen.

Ein harter, planer Gegenstand (z. B. ein Buch) wird in exakt 15 cm Entfernung parallel zur Sensorfront platziert. Anschließend wird der Serielle Monitor der Arduino IDE mit einer Baudrate von 115200 geöffnet. Der ausgegebene Wert muss durch den Aufruf der Funktion `sonar.ping_median(5)` sehr stabil bei 15 cm liegen. Die Schwankung darf dabei die gerätespezifische Auflösung des Moduls von 3 mm [KT-12] kaum überschreiten. Das Skript bestätigt somit, dass das Sende- und Empfangsmodul korrekt arbeiten und die serielle Datenübertragung fehlerfrei funktioniert.

9.9.2. Alle Funktionen testen

Ein vollständiger Systemtest kann wie folgt durchgeführt werden. Hierfür können die Ausgaben des Seriellen Monitors aus dem vorherigen Testcode 9.1 oder das Verhalten der LEDs aus dem Parkassistenten-Code 9.3 beobachtet werden:

- **Minimaldistanz:** Das Objekt wird auf unter 2 cm an den Sensor herangeführt. Da der Sensor nach dem Senden des 200 µs langen Bursts eine kurze Umschaltzeit benötigt [KT-12], kann er Echos im absoluten Nahbereich physikalisch nicht mehr auflösen. Der Monitor sollte hier durch die Bibliothek korrekterweise Außerhalb der Reichweite (Rückgabewert 0) ausgeben [Eck23].
- **Maximaldistanz:** Der Sensor wird in einen leeren, großen Raum gerichtet, sodass das nächste Hindernis mehr als 300 cm entfernt ist. Laut Spezifikation verweilt der ECHO-Pin bei einem fehlenden Echo für 200 ms auf einem HIGH-Pegel [KT-12]. Der Test belegt, dass die NewPing-Bibliothek diesen Hardware-Timeout sicher abfängt, ohne dass sich der Mikrocontroller in einer blockierenden Endlosschleife aufhängt [Eck23].
- **Winkeltest (Abstrahlkegel):** Ein Objekt wird in 50 cm Entfernung positioniert und langsam seitlich aus dem Zentrum bewegt. Überschreitet der Winkel 15° zur Mittelachse, wird die Fläche vom Objekt für die Ultraschallwellen zu klein [KT-12]. Die Messwerte brechen ab, was die seitlichen Grenzen des Erfassungsbereichs bestätigt.
- **Dynamischer Reaktionstest:** Mit dem Code des automatisierten Parkassistenten aus Abschnitt 9.3 wird die Reaktionsgeschwindigkeit der Gesamtanlage getestet. Ein Objekt wird zügig auf den Sensor zubewegt. Die gelbe Warn-LED muss exakt beim Unterschreiten der 50-cm-Marke aufleuchten, und der Buzzer muss ohne spürbare Software-Verzögerung bei exakt 10 cm triggern.

9.10. Weiterführende Literatur

SparkFun Electronics / ElecFreaks (2019): HC-SR04 Ultrasonic Sensor Datasheet, Verfügbar unter: <https://www.sparkfun.com/datasheets/Sensors/Ultrasonic/HCSR04.pdf> [abgerufen am 28.04.2026]

- Diese technische Spezifikation bildet die fundamentale Grundlage für die Ansteuerung. Sie enthält die exakten Timing-Diagramme für das Trigger- und Echo-Signal sowie Angaben zum Öffnungswinkel des Schallkegels, die für die Gehäuseplanung wichtig sind.

Eckel, Tim (2023): Arduino NewPing Library Documentation, GitHub/Bitbucket Repository, Verfügbar unter: <https://bitbucket.org/teckel12/arduino-newping/wiki/Home> [abgerufen am 28.04.2026]

- Eine umfassende Dokumentation zur verwendeten Programmbibliothek NewPing Library von Tim Eckel. Es werden die Vorteile gegenüber herkömmlichen Messmethoden erläutert und die Implementierung von Filtern wie der `ping_median()`-Funktion zur Rauschunterdrückung detailliert beschrieben.

Ekbert Hering und Gert Schönfelder, Sensoren in Wissenschaft und Technik Funktionsweise und Einsatzgebiete, Springer Vieweg, 2023, ISBN: 9783658394905 [HS23]

- Eine umfassende und aktuelle Einführung in die Grundlagen und Anwendungen von Sensoren in Technik, Biologie und Medizin. Das Buch behandelt neben physikalischen Grundlagen auch Themen wie Schnittstellen, Sicherheit und Signalverarbeitung bei Ultraschallwandlern.

10. SG90 Servomotor

Einleitung

Dieses Kapitel beschreibt den mechanischen Aktor des Systems, den Micro-Servomotor TowerPro SG90. Es werden die technischen Spezifikationen, die hardwareseitige Einbindung sowie die softwareseitige Ansteuerung über die Standard-Bibliotheken detailliert erläutert.

10.1. Allgemeines

Ein Servomotor ist ein geschlossenes mechatronisches System (Closed-Loop), das für präzise Winkelpositionierungen eingesetzt wird. Im Gegensatz zu kontinuierlich drehenden Gleichstrommotoren besteht ein Standard-Servo aus einem DC-Motor, einem Untersetzungsgetriebe, einem Potentiometer zur Positionserfassung und einer Steuerelektronik. Der Mikrocontroller sendet ein Pulsweitenmodulations-Signal (*PWM*), welches die Steuerelektronik auswertet und den Motor exakt auf den gewünschten Winkel dreht und dort aktiv hält. Servomotoren findet man vor allem im RC-Modellbau (z.B. Lenkung, Ruder, Klappen), in Robotergelenken sowie in Schwenkmechanismen für Kameras oder Sensoren, bei denen wiederholgenaue und stabile Positionen erforderlich sind.[Mik20]

10.2. Servomotor TowerPro SG90

Der TowerPro SG90 ist einer der weltweit am häufigsten verwendeten Micro-Servomotoren für kleine Projekte. Er zeichnet sich durch sein extrem geringes Gewicht, seine kompakte Bauform und sein Nylon-Zahnradgetriebe aus. Der Aktor verfügt über ein dreipoliges Anschlusskabel: Braun für die Masse (*GND*), Rot für die Versorgungsspannung (*VCC*) und Orange für das PWM-Steuersignal (*Signal*). Der TowerPro SG90 hat einen mechanischen Rotationsspielraum von 180 Grad (0° bis 180°). Gesteuert wird er über ein 50-Hz-PWM-Signal (20 ms Periodendauer). Eine Pulsweite von 1 ms entspricht theoretisch 0°, 1,5 ms entsprechen 90° (Mittelstellung) und 2 ms entsprechen 180°.[Pat; Com17]

10.3. Spezifikation

- **Quellen referenzieren:** Die physikalischen und elektrischen Kennwerte beziehen sich auf technische Angaben zum SG90 von Elektronik-Kompendium und Components101.[Pat; Com17]
- **Extrahierte Informationen:**
 - Betriebsspannung: 4,8 V (ca. 5 V)
 - Stellmoment (Stall Torque): 1,8 kg/cm (bei 4,8 V)
 - Stellgeschwindigkeit (ohne Last): 0,1 Sekunden / 60 Grad (bei 4,8 V)
 - Gewicht: 9 Gramm
 - Getriebetyp: Kunststoff (Nylon)

- Steuersignal: PWM (50 Hz / 20 ms Periode)
- Dimensionen: ca. 22,2 x 11,8 x 31 mm

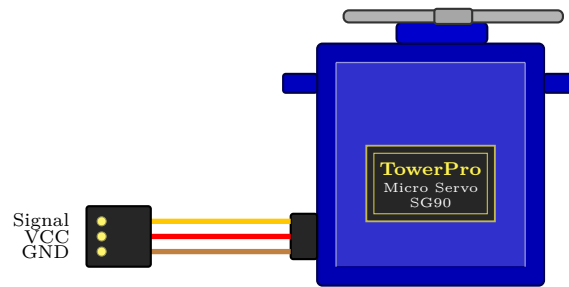


Abbildung 10.1.: Schematische Darstellung des Servomotors TowerPro SG90

- **Schaltplan:** Das rote Kabel (*VCC*) des SG90 wird mit dem 5V-Ausgang (*VUSB*) des Systems verbunden, um ausreichend Leistung für den Motor bereitzustellen. Das braune Kabel (*GND*) wird mit der gemeinsamen Masse verbunden. Das orange Kabel (*Signal*) wird an einen digitalen PWM-fähigen Pin (z.B. Pin 9) des Mikrocontrollers angeschlossen.[Pat]

Hinweis: Der Arduino Nano 33 BLE Sense arbeitet mit 3,3V-I/O-Pegeln und ist nicht 5V-tolerant.[Ard26a] Viele Servomotoren können jedoch ein 3,3V-PWM-Steuersignal verarbeiten, sofern der Servo selbst mit einer geeigneten Versorgungsspannung betrieben wird und eine gemeinsame Masse vorhanden ist.[Dr]

10.4. Bibliothek

10.4.1. Beschreibung

Für die Ansteuerung des SG90 wird in der Arduino-Umgebung die integrierte *Servo*-Bibliothek genutzt[Lib25a]. Dadurch entfällt für den Programmierer die aufwändige Erzeugung des exakten 50-Hz-PWM-Signals, und der Motor kann bequem über die direkte Vorgabe eines Winkelwertes von 0 bis 180 Grad angesteuert werden.[Lib25a]

10.4.2. Installation

- **Installation:** Die Bibliothek kann in der Arduino IDE über den Bibliotheksverwalter eingebunden werden. Im Programm wird sie anschließend mit `#include <Servo.h>` importiert.[Mik20]
- **Data types:** Die Zielwinkel werden als `int` (Integer/Ganzzahlen) übergeben. Für präzisere Steuerungen können Pulsweiten in Mikrosekunden ebenfalls als `int` übergeben werden (vgl. [Lib25b]).
- **Data structure:** Jeder Servomotor wird im Code als Objekt der Klasse *Servo* deklariert (z.B. `meinServo;`).
- **Data quantity:** Auf Standard Arduino-Boards unterstützt die Bibliothek bis zu 12 Servos gleichzeitig; auf speziellen Architekturen (wie der Mbed-OS Architektur des Nano 33 BLE) kann dies variieren, reicht jedoch für mehrere Aktoren vollkommen aus.[Doc26]
- **Data quality:** Die Bibliothek arbeitet timer- und interruptbasiert. Die Servosignale werden im Hintergrund mit dem zuletzt über `write()` gesetzten Wert erzeugt, wodurch keine manuelle PWM-Erzeugung mit blockierenden Delay-Schleifen notwendig ist.[Lib25b]

10.4.3. Funktionen

Die wichtigsten Methoden der Bibliothek sind:

- `attach(pin)`: Verbindet das Servo-Objekt mit einem bestimmten digitalen Pin.[Ard26b]
- `write(angle)`: Dreht den Servo auf den entsprechenden Winkel (0° bis 180°).
- `writeMicroseconds(uS)`: Ermöglicht die Ansteuerung über Pulsweiten in Mikrosekunden.[Ard26d]
- `read()`: Gibt den zuletzt gesetzten Servo-Winkel zurück.[Ard26c]

10.5. Beispiel

Dieser Abschnitt zeigt ein elementares Beispiel „Sweep“, bei dem der Servo kontinuierlich von 0 auf 180 Grad und wieder zurück schwenkt.

10.5.1. Handbuch

Schließen Sie das rote Kabel an 5V, das braune an GND und das orange Signalkabel an Pin 9 des Arduinos an. Kompilieren Sie das Programm und laden Sie es auf das Board. Der Arm des Servomotors sollte nun anfangen, langsam und gleichmässig hin und her zu wischen.

10.5.2. An einem Beispiel

Das Skript in 10.1 bindet zunächst die Bibliothek `Servo.h` ein und erstellt mit `radarServo` ein Servo-Objekt. Zusätzlich wird die Variable `pos` zur Speicherung der aktuellen Winkelposition verwendet. In der `setup()`-Routine wird der Motor über `radarServo.attach(9)` an den digitalen Pin 9 gebunden.

In der `loop()`-Schleife sorgen zwei `for`-Schleifen für die kontinuierliche Schwenkbewegung des Servomotors. Die erste Schleife erhöht den Winkel schrittweise von 0° auf 180°. Dabei wird mit `radarServo.write(pos)` jeweils eine absolute Zielposition angefahren. Nach jeder Winkeländerung folgt mit `delay(15)` eine kurze Pause von 15 ms, damit der Motor die physikalische Position erreichen kann. Die zweite Schleife führt anschließend die Rückwärtsbewegung von 180° zurück auf 0° aus. Dadurch entsteht eine fortlaufende Hin- und Herbewegung des Servomotors.

10.5.3. Code

Listing 10.1.: Beispiel Code für den TowerPro SG90

```

1000 /**
1001  * @file    Servomotor.ino
1002  * @author  Simon Müller
1003  * @date    2026-05-01
1004  * @version 1.0
1005  *
1006  * @brief   Servo-based radar scanning system.
1007  *
1008  * This sketch controls a TowerPro SG90 (or similar) servo to perform
1009  * a continuous back-and-forth sweep from 0 deg to 180 deg and back.
1010  * All position commands are absolute angles
1011  * (0180 deg range).
1012  */
1013
1014 #include <Servo.h>

```

```

1016 Servo radarServo;    ///< Servo object used for radar scanning
1017 int pos = 0;         ///< Current servo position in degrees (0180,
1018                       absolute)
1019
1020 /**
1021  * @brief Initializes the servo on startup.
1022  *
1023  * Attaches the servo object to digital pin 9 and prepares it
1024  * for subsequent position commands. The servo will interpret
1025  * all following write() calls as absolute target angles in
1026  * the range 0°180°.
1027  */
1028 void setup() {
1029     radarServo.attach(9); // Attach servo to pin 9
1030 }
1031
1032 /**
1033  * @brief Main loop for continuous servo motion.
1034  *
1035  * The servo is moved in two phases using absolute angle commands:
1036  * - Phase 1: Sweep from 0° to 180° in 1° steps
1037  * - Phase 2: Sweep from 180° back to 0° in 1° steps
1038  *
1039  * Each call to radarServo::write(pos) sets an absolute target
1040  * angle; it does NOT move the servo by a relative offset.
1041  * With a 15 ms delay between steps, one complete cycle
1042  * takes approximately 5.4 seconds.
1043  */
1044 void loop() {
1045     // Phase 1: sweep from 0° to 180° (absolute positions)
1046     for (pos = 0; pos <= 180; pos += 1) {
1047         radarServo.write(pos); ///< Set servo to absolute angle 'pos'
1048         delay(15);             ///< Wait 15 ms to reach the position
1049     }
1050
1051     // Phase 2: sweep from 180° back to 0° (absolute positions)
1052     for (pos = 180; pos >= 0; pos -= 1) {
1053         radarServo.write(pos); ///< Set servo to absolute angle 'pos'
1054         delay(15);             ///< Wait 15 ms
1055     }
1056 }

```

../Code/Servomotor/Servomotor.ino

10.5.4. Dateien

Für das Ausführen des Codes wird ausschließlich die Standard-Headerdatei `Servo.h` benötigt, welche in der Arduino IDE vorinstalliert ist.

10.6. Kalibrierung

Bei Servomotoren können die praktisch nutzbaren Endpositionen von den theoretischen PWM-Werten abweichen. Wird der Motor mit zu kleinen oder zu großen Pulsweiten an seine mechanischen Grenzen gefahren, kann dies zu Brummen, Vibrationen oder mechanischer Belastung führen[Ard26d]. Zur Kalibrierung bietet die `attach()`-Funktion optionale Parameter für die minimalen und maximalen Mikrosekunden: `radarServo.attach(9, 500, 2400)`. Diese Werte können durch iteratives Ausprobieren mit `writeMicroseconds()` an den individuellen Motor angepasst werden, bis der gewünschte Winkelbereich ohne mechanisches Blockieren erreicht ist.

10.7. Einfacher Code (ohne Bibliothek)

Der Code in 10.2 erzeugt das PWM-Signal für den Servo manuell ohne die Verwendung der Servo-Bibliothek. In der `setup()`-Routine wird Pin 9 mit `pinMode(9, OUTPUT)` als Ausgang konfiguriert. Die `loop()`-Funktion generiert anschließend ein 50-Hz-Signal. Dazu wird der Pin mit `digitalWrite(9, HIGH)` für 1500 µs auf HIGH gesetzt, was ungefähr einer Servo-Position von 90° entspricht. Danach wird der Pin mit `digitalWrite(9, LOW)` für 18500 µs auf LOW gesetzt, sodass eine vollständige Periodendauer von 20 ms entsteht.

Diese Methode zeigt vereinfacht, wie die Ansteuerung eines Servomotors über ein PWM-Signal funktioniert. Da der Ablauf jedoch durch die verwendeten Verzögerungen blockierend ist, eignet sich diese Variante nur eingeschränkt für komplexere Anwendungen. In solchen Fällen sollte die Servo-Bibliothek verwendet werden, da diese mit Timern arbeitet und die Ansteuerung komfortabler übernimmt.

Listing 10.2.: Code ohne Bibliothek für den TowerPro SG90

```

1000 /**
1001  * @file ServoManual.ino
1002  * @brief Manual servo control without library
1003  *
1004  * @details Demonstrates servo motor control through direct PWM
1005  *          generation
1006  * using digitalWrite() and delayMicroseconds(). The servo is positioned
1007  * at approximately 90° (center position). This method shows how the
1008  * Servo library works at the hardware level.
1009  *
1010  * @author Simon Müller
1011  * @date 01.05.2026
1012  * @version 1.0
1013  *
1014  * @section hardware Hardware
1015  * – Arduino Board (e.g. Uno, Nano)
1016  * – Servo motor connected to Pin 9
1017  * – Power supply: 5V for servo (VCC and GND)
1018  *
1019  * @section notes Notes
1020  * This implementation uses blocking delays and is for educational
1021  * purposes.
1022  * For production code, use the Arduino Servo library instead.
1023  */
1024
1025 int servoPin = 9; ///< Pin number for servo control signal
1026
1027 /**
1028  * @brief Initialization at program startup
1029  *
1030  * @details Configures the servo pin as OUTPUT to enable
1031  * sending PWM signals to the servo motor.
1032  *
1033  * @return void
1034  */
1035 void setup() {
1036     pinMode(servoPin, OUTPUT); // Set servo pin as output
1037 }
1038
1039 /**
1040  * @brief Main loop for continuous PWM generation
1041  *
1042  * @details Manually generates a 50 Hz PWM signal (20 ms period).
1043  * The HIGH pulse of 1.5 ms corresponds to a servo position of approx.
1044  * 90°.
1045  *
1046  * PWM Timing:
1047  * – HIGH phase: 1500 µs (1.5 ms) 90° position (center)

```

```

1046 * - LOW phase: 18500  $\mu$ s (18.5 ms) fills up to 20 ms total period
1047 * - Frequency: 50 Hz (1000000  $\mu$ s / 20000  $\mu$ s = 50 Hz)
1048 *
1049 * @note This method blocks program execution. For more complex
1050 * applications, the Servo library should be used.
1051 *
1052 * @return void
1053 */
1054 void loop() {
1055     // Move to approx. 90 degrees (1.5ms HIGH pulse)
1056     digitalWrite(servoPin, HIGH);    ///< Starts the HIGH pulse
1057     delayMicroseconds(1500);          ///< Pulse width: 1.5 ms 90°
1058     position
1059     digitalWrite(servoPin, LOW);      ///< Ends the HIGH pulse
1060
1061     // Fill the remaining 20ms period (50 Hz)
1062     delayMicroseconds(18500);         ///< LOW phase: 18.5 ms until next
1063     period
1064 }

```

../Code/Servomotor/ServomotorohneBib.ino

10.8. Einfache Anwendung

Eine klassische Anwendung für den SG90 ist die Basis eines **Ultraschall-Radarsystems**. Der Ultraschallsensor HC-SR04 wird auf der Achse des Servos montiert. Der Servo tastet systematisch in 1-Grad-Schritten die Umgebung vor sich ab (0° bis 180°). Bei jedem Schritt wird eine Entfernungsmessung durchgeführt. Diese Kombination aus Aktor-Winkel und Sensor-Distanz ermöglicht es der Software (z. B. in Processing), eine zweidimensionale „Radarkante“ der Umgebung auf dem Bildschirm zu zeichnen.[die25]

10.9. Tests

10.9.1. Einfacher Funktionstest

Mit dem „Sweep“-Code aus Abschnitt 10.2 lässt sich prüfen, ob der Motor korrekt verkabelt ist. Der Motor sollte möglichst ruckelfrei und gleichmäßig schwenken. Ein stotternder Motor oder ein Neustart des Arduinos während der Bewegung kann auf eine unzureichende Stromversorgung, beispielsweise einen VCC-Abfall, hinweisen.

10.9.2. Alle Funktionen testen

Ein vollständiger Systemtest umfasst:

1. **Grenzttest:** Anfahren von 0° und 180°. Der Motor darf in diesen Endlagen nicht surren oder ungewöhnlich warm werden.[Ard26d]
2. **Geschwindigkeitstest:** Befehle `write(0)` und sofort danach `write(180)` ohne Delay. Messung, ob der Motor die physikalische Herstellerangabe von 0,3 Sekunden für diesen 180°-Weg einhalten kann.[Com17]
3. **Präzisionstest:** Verwendung von `writeMicroseconds(1500)` zur exakten Zentrierung, gefolgt von der Überprüfung der Winkelgenauigkeit mittels eines Geodreiecks auf dem Rotorkopf.[Ard26d]

10.10. Weiterführende Literatur

Teil IV.

Domain Knowledge - Tools

11. Python Tool XY

11.1. Mikrocontroller-Programmierung mit der Arduino IDE

11.1.1. Allgemeines zur Arduino IDE

Die Arduino Integrated Development Environment (IDE) ist eine plattformunabhängige Open-Source-Entwicklungsumgebung, die speziell für die Programmierung von Mikrocontrollern entworfen wurde. Sie basiert im Kern auf der Programmiersprache C/C++, verbirgt jedoch viele komplexe hardwarenahe Konfigurationen hinter einer stark vereinfachten Syntax und einer umfangreichen Standardbibliothek.

Ein klassisches Arduino-Programm (auch „Sketch“ genannt) ist strukturell immer in zwei wesentliche Hauptfunktionen unterteilt: Die `setup()`-Funktion, welche genau einmal beim Systemstart ausgeführt wird und für die Initialisierung von Pins und Schnittstellen zuständig ist, sowie die `loop()`-Funktion, welche im Anschluss als Endlosschleife die eigentliche Arbeitslogik des Mikrocontrollers abarbeitet. Die fertigen Programme werden direkt aus der IDE heraus kompiliert und über eine serielle USB-Verbindung auf den Flash-Speicher des Mikrocontrollers übertragen.

11.1.2. Das Steuerungsprogramm des Ultraschallradars

Der Quellcode für den Arduino Nano 33 BLE übernimmt die direkte Hardwaresteuerung der Sensorik, der Aktuatorik sowie der visuellen Statusanzeigen (LEDs). Bevor das System in den regulären Betriebsmodus wechselt, durchläuft es in der `setup()`-Phase einen sogenannten Power-On Self-Test (POST). Hierbei teilt der Mikrocontroller der verbundenen Processing-Software zunächst den Status `S:TESTING` mit. Daraufhin werden die LEDs auf Funktion geprüft, der Servomotor absolviert eine Testdrehung und der Ultraschallsensor führt eine initiale Referenzmessung durch.

Nur wenn diese Testmessung valide Werte (zwischen 1 cm und 400 cm) liefert, wird das System freigegeben, die grüne Status-LED aktiviert und das Signal `S:OK` an den Computer gesendet. Tritt ein Fehler auf, beispielsweise durch einen Verbindungsabbruch zum Sensor (Timeout), leuchtet die rote LED auf, das System blockiert aus Sicherheitsgründen den weiteren Betrieb und meldet `S:ERR_SENSOR`.

Im laufenden und fehlerfreien Betrieb (`loop`) iteriert das System kontinuierlich durch die Positionen des Servomotors (0° bis 180° und zurück). Nach jedem Positionsschritt pausiert das System kurz, um die mechanische Trägheit des Motors auszugleichen. Anschließend feuert der HC-SR04-Sensor einen 10 µs langen Ultraschall-Impuls ab und misst die Zeit bis zum Eintreffen des Echos. Aus dieser Laufzeit errechnet der Mikrocontroller unter Einbeziehung der Schallgeschwindigkeit die genaue Distanz.

Abschließend formatiert der Arduino diese Wertepaare in das speziell für dieses Projekt definierte Kommunikationsprotokoll (`D:Winkel,Distanz`) und übermittelt sie in Echtzeit an die serielle Schnittstelle zur weiteren grafischen Auswertung.

11.1.3. Quellcode der Arduino-Steuerung

Listing 11.1: Arduino Steuerungscode mit Doxygen-Dokumentation

```

1  /**
2   * @file UltraschallradarArduinoIDeMitDoxygen.ino
3   * @brief Firmware für ein automatisches 120-Grad-Ultraschallradar.
4   * @details Diese Software steuert ein Ultraschallradar, bestehend aus einem
5   * Arduino Nano V3, einem HC-SR04 Ultraschallsensor und einem SG90 Positions-Servomotor.
6   * Beim Systemstart wird eine Zentrierung auf 90° durchgeführt und ein visueller
7   * Power-On Self-Test (POST) über zwei Status-LEDs signalisiert. Im laufenden Betrieb
8   * wird ein kontinuierlicher Schwenk von 30° bis 150° ausgeführt, die gemessene
9   * Distanz ermittelt und über das serielle Protokoll an das Processing-Frontend gesendet.
10  * * @author Wilko Hinrichs
11  * @date 2026-06-08
12  * @version 1.1
13  */
14
15  #include <Servo.h>
16
17  /* --- PIN-DEFINITIONEN --- */
18  const int PIN_LED_GREEN = 2;    ///< GPIO-Pin für die grüne Status-LED (System betriebsbereit)
19  const int PIN_LED_RED = 3;      ///< GPIO-Pin für die rote Status-LED (Fehler / Selbsttest)
20  const int PIN_TRIG = 4;         ///< Digitaler Ausgangs-Pin zur Initiierung des Ultraschall-Impulses (Trig)
21  const int PIN_ECHO = 5;         ///< Digitaler Eingangs-Pin zur Messung der Schall-Laufzeit (Echo)
22  const int PIN_SERVO = 9;        ///< PWM-Ausgangs-Pin zur Winkelsteuerung des SG90 Servomotors
23
24  /* --- GLOBALE OBJEKTE --- */
25  Servo radarServo;               ///< Servo-Instanz der offiziellen Arduino-Bibliothek zur Achsensteuerung.
26
27  /**
28   * @brief Initialisiert die Hardware-Peripherie und führt den Boot-Selbsttest aus.
29   * @details Konfiguriert die seriellen Kommunikationsparameter auf 9600 Baud, definiert
30   * die Pin-Modi (INPUT/OUTPUT) und leitet den Selbsttest ein, bei dem zunächst beide LEDs leuchten. Der Se
31   * erfolgreiche Testmessung im gültigen physikalischen Bereich (0 - 400 cm) registriert,
32   * schaltet das System die rote LED aus und startet den Normalbetrieb. Andernfalls blockiert
33   * das System im Fehlermodus.
34   */
35  void setup() {
36      // Serielle Schnittstelle mit stabiler Standard-Baudrate initialisieren
37      Serial.begin(9600);
38
39      // Definition der Signalrichtungen
40      pinMode(PIN_LED_GREEN, OUTPUT);
41      pinMode(PIN_LED_RED, OUTPUT);
42      pinMode(PIN_TRIG, OUTPUT);
43      pinMode(PIN_ECHO, INPUT);
44
45      // SELBSTTEST-BEGINN: Beide LEDs signalisieren den Testzustand
46      Serial.println("S:TESTING");
47      digitalWrite(PIN_LED_RED, HIGH);
48      digitalWrite(PIN_LED_GREEN, HIGH);
49
50      // ZENTRIERUNG: Servo fährt in die exakte Mitte (90 Grad) für die Gehäusemontage
51      radarServo.attach(PIN_SERVO);
52      radarServo.write(90);
53      delay(1500); // Ausreichend Zeit, um den Sensorkopf mechanisch auszurichten
54
55      // Ultraschall-Testmessung triggern (10 Mikrosekunden High-Impuls)
56      digitalWrite(PIN_TRIG, LOW);
57      delayMicroseconds(2);
58      digitalWrite(PIN_TRIG, HIGH);
59      delayMicroseconds(10);
60      digitalWrite(PIN_TRIG, LOW);
61
62      // Messung der Echo-Rücklaufzeit mit einem Timeout von 30ms (ca. 5 Meter Reichweite)
63      long duration = pulseIn(PIN_ECHO, HIGH, 30000);
64      float distance = duration * 0.0343 / 2.0;

```

```

65
66 // Validierung des Sensorsignals
67 if (distance > 0 && distance < 400) {
68     // SELBSTTEST ERFOLGREICH: Rot aus, Grün bleibt an -> Normalbetrieb
69     digitalWrite(PIN_LED_RED, LOW);
70     digitalWrite(PIN_LED_GREEN, HIGH);
71     Serial.println("S:OK");
72 } else {
73     // SELBSTTEST FEHLGESCHLAGEN: Grün aus, Rot bleibt an -> System sperren
74     digitalWrite(PIN_LED_GREEN, LOW);
75     digitalWrite(PIN_LED_RED, HIGH);
76     Serial.println("S:ERR_SENSOR");
77     while(1); // Endlosschleife verhindert die Ausführung der loop() bei Defekt
78 }
79 }
80
81 /**
82  * @brief Kontinuierliche Hauptschleife für die periodische Schwenkbewegung.
83  * @details Realisiert die Schwenkbewegung des Radars symmetrisch um die Mittelachse (90°).
84  * Der gesamte Scanbereich beträgt exakt 120 Grad und wird in drei Phasen durchlaufen:
85  * 1. Schwenk von der Mitte (90°) nach ganz links zur Grenze (30°).
86  * 2. Kompletter Schwenk von ganz links (30°) nach ganz rechts zur Grenze (150°).
87  * 3. Rückschwenk von ganz rechts (150°) zurück in die Ausgangsmitte (90°).
88  */
89 void loop() {
90     // Phase 1: Von der Mitte nach links (90° bis 30°)
91     for (int angle = 90; angle >= 30; angle--) {
92         scanPosition(angle);
93     }
94
95     // Phase 2: Von links komplett nach rechts (30° bis 150°)
96     for (int angle = 30; angle <= 150; angle++) {
97         scanPosition(angle);
98     }
99
100    // Phase 3: Von rechts wieder zurück in die Mitte (150° bis 90°)
101    for (int angle = 150; angle >= 90; angle--) {
102        scanPosition(angle);
103    }
104 }
105
106 /**
107  * @brief Führt einen spezifischen Winkel an, misst die Distanz und überträgt das Datenprotokoll.
108  * @details Diese Funktion kapselt den gesamten Messvorgang für ein Gradsegment:
109  * 1. Der Servo erhält den neuen Stellbefehl. Ein Delay von 35ms sichert der Mechanik
110  * des SG90 ausreichend Zeit zu, die Position vibrationsfrei zu erreichen.
111  * 2. Der HC-SR04 sendet einen Ultraschall-Burst aus.
112  * 3. Über die Schallgeschwindigkeit in Luft ( $\frac{343 \text{ m/s}}{2}$ ) wird aus der Laufzeit
113  * die Distanz in cm berechnet ( $\text{Weg} = \frac{\text{Zeit}}{2} \times 0.0343$ ).
114  * 4. Die Daten werden im standardisierten Stringformat 'D:[Winkel],[Distanz]' über die
115  * serielle Schnittstelle (UART) an Processing übertragen.
116  * * @param angle Der präzise anzufahrende Zielwinkel in Grad (30° bis 150°).
117  */
118 void scanPosition(int angle) {
119     radarServo.write(angle);
120     delay(35); // Einschwingzeit für die Servomechanik (wichtig gegen Bildzittern)
121
122     // Ultraschall-Messzyklus starten
123     digitalWrite(PIN_TRIG, LOW);
124     delayMicroseconds(2);
125     digitalWrite(PIN_TRIG, HIGH);
126     delayMicroseconds(10);
127     digitalWrite(PIN_TRIG, LOW);
128
129     // Pulsdauer am Echo-Pin messen
130     long duration = pulseIn(PIN_ECHO, HIGH, 30000);
131     int distance = duration * 0.0343 / 2.0;
132
133     // Protokollausgabe an das Processing-Frontend (z.B. "D:45,18")

```

```
134   Serial.print("D:");  
135   Serial.print(angle);  
136   Serial.print(",");  
137   Serial.println(distance);  
138 }
```

12. Hardware Tool XY

12.1. Benutzeroberfläche und Visualisierung mit Processing

12.1.1. Allgemeines zu Processing

Processing ist eine Open-Source-Programmiersprache und integrierte Entwicklungsumgebung (IDE), die ursprünglich für die Bereiche visuelle Kunst, Design und elektronische Medien entwickelt wurde. Sie basiert auf der Programmiersprache Java, bietet jedoch eine stark vereinfachte Syntax und eine speziell auf grafische Anwendungen zugeschnittene Bibliothek. Dadurch ermöglicht Processing es, komplexe visuelle Ausgaben, Animationen und Interaktionen mit nur wenigen Zeilen Code zu realisieren. Ein zentraler Vorteil von Processing im Kontext von Mikrocontroller-Projekten ist die native und unkomplizierte Unterstützung der seriellen Kommunikation. Dies macht es zu einem idealen grafischen Frontend für Arduino-Anwendungen. Während der Mikrocontroller die zeitkritische Hardware-Steuerung und Sensordaten-Erfassung übernimmt, kann Processing diese Rohdaten am Computer in Echtzeit auswerten und grafisch aufbereiten.

12.1.2. Das Ultraschallradar-Programm

Für das vorliegende Projekt wird Processing genutzt, um die vom Arduino Nano 33 BLE gesendeten Sensordaten (Winkel und Distanz) abzufangen und in eine klassische Radar-Anzeige zu übersetzen. Die Software arbeitet dabei als Zustandsmaschine (State Machine) mit den Zuständen **TESTING**, **OK** und **ERROR**. Während der Arduino beim Systemstart kalibriert wird, zeigt die Benutzeroberfläche einen Ladebildschirm an. Erst wenn das Signal **S:OK** über die serielle Schnittstelle empfangen wird, wechselt die Software in den Live-Scan-Modus.

Die Visualisierung simuliert das Nachleuchten eines echten Radars durch das wiederholte Zeichnen eines teiltransparenten Hintergrunds. Das Interface besteht aus einem Koordinatensystem mit konzentrischen Ringen, welche die Entfernungen (10 cm bis 40 cm) repräsentieren, sowie Hilfslinien für die verschiedenen Winkel. Die aktuelle Ausrichtung des Servomotors wird als grüne Scan-Linie dargestellt.

Sobald ein Objekt innerhalb der definierten Reichweite von 40 cm erkannt wird, transformiert das Programm die Polarkoordinaten (Winkel und Distanz) in kartesische Koordinaten (X/Y-Pixelwerte). Das Hindernis wird daraufhin als roter Punkt im Raster eingezeichnet. Zusätzlich generiert die Software eine auffällige, blinkende Warnmeldung („OBJEKT ERKANNT!“) sowie eine dynamische Textausgabe, um den Nutzer visuell auf das Hindernis hinzuweisen.

Um die genaue Funktionsweise und Struktur der Software nachvollziehbar zu machen, ist der Doxygen-dokumentierte Quellcode im Folgenden eingeblendet.

12.1.3. Quellcode der Processing-Anwendung

Listing 12.1: Processing Benutzeroberfläche

```

1  import processing.serial.*;
2
3  /**
4   * @file BenutzeroberflaecheMitDoxygen.pde
5   * @brief Grafische Benutzeroberfläche (Frontend) für ein 120-Grad-Ultraschallradar.
6   * @details Diese Software empfängt Messdaten (Winkel und Distanz) in Echtzeit über die
7   * serielle Schnittstelle von einem Arduino Nano V3. Die Daten werden gefiltert,
8   * mittels einer intelligenten Kantenerkennung mathematisch von der physikalischen
9   * Schallkegel-Streuung bereinigt und auf einem animierten Sonar-Bildschirm dargestellt.
10  * * @author Wilko Hinrichs
11  * @date 2026-06-08
12  * @version 1.2
13  */
14
15  /* --- GLOBALE OBJEKTE & STATUSVARIABLEN --- */
16  Serial myPort;          ///< Das serielle Schnittstellen-Objekt zur Hardware-Kommunikation.
17  String systemState = "TESTING";  ///< Aktueller Systemstatus des Radars ("TESTING", "OK", "ERR_SENSOR", "ERR_
18  String errorMessage = "";      ///< Klartext-Fehlermeldung bei Hardware- oder Portproblemen.
19
20  float angle = 90;          ///< Der aktuell vom Arduino übermittelte Schwenkwinkel (30° bis 150°).
21  float distance = 0;        ///< Die aktuell gemessene Entfernung des Hindernisses in Zentimetern.
22
23  /* --- VISUALISIERUNGS-EINSTELLUNGEN --- */
24  int radarRadius = 400;     ///< Maximaler Zeichnungsradius des Sonar-Rasters in Pixeln.
25  int maxDistanceCm = 40;    ///< Skalierungsgrenze: Maximale Messdistanz des Radars in Zentimetern.
26
27  /** * @brief Speicher-Array für das visuelle "Nachleuchten" des Radars.
28   * @details Der Index entspricht dem Winkel (0-180°). Der gespeicherte Wert ist die Distanz in cm.
29   */
30  float[] radarWerte = new float[181];
31
32  /**
33   * @brief Initialisiert das Anwendungsfenster und konfiguriert die serielle Kommunikation.
34   * @details Das Fenster wird auf eine feste Auflösung gesetzt, das Antialiasing (smooth) aktiviert
35   * und versucht, die Verbindung zum USB-Treiber des Arduino Nano mit 9600 Baud aufzubauen.
36   */
37  void setup() {
38      size(1000, 650);
39      smooth();
40
41      // Initialisiere das Daten-Array mit Nullen (keine Hindernisse beim Start)
42      for (int i = 0; i <= 180; i++) {
43          radarWerte[i] = 0;
44      }
45
46      // Verbindungsaufbau zur Hardware
47      try {
48          String portName = "/dev/cu.usbserial-A106PYCP";
49          myPort = new Serial(this, portName, 9600);
50          myPort.bufferUntil('\n'); // Event wird erst bei vollständiger Zeile ausgelöst
51      } catch (Exception e) {
52          systemState = "ERR_PORT";
53          errorMessage = "Port_nicht_gefunden!";
54      }
55  }
56
57  /**
58   * @brief Hauptzeichenschleife (Render-Engine) der Benutzeroberfläche.
59   * @details Draw wird ununterbrochen ausgeführt und zeichnet das Radar-Layout in fünf Schichten:
60   * 1. Das kreisförmige EchoLot-Raster inklusive Zentimeter-Angaben.
61   * 2. Die radialen Richtungs-Vektoren inklusive Gradzahlen.
62   * 3. Die gefilterten Objekt-Cluster (Echtzeit-Hindernisse).
63   * 4. Den rotierenden, leuchtenden Radar-Abtaststrahl.
64   * 5. Das mathematische Status-Dashboard in der oberen linken Ecke.

```

```

65  */
66  void draw() {
67      background(6, 12, 6); // Dunkles Sci-Fi-Radar-Grün
68
69      // Verschiebe den Koordinatenursprung (0,0) in das untere Zentrum des Fensters
70      pushMatrix();
71      translate(width/2, height - 70);
72
73      // =====
74      // SCHRITT 1: GRID-KREISE ZEICHNEN & BESCHRIFTEN
75      // =====
76      strokeWeight(1);
77      for (int r = 100; r <= radarRadius; r += 100) {
78          stroke(0, 100, 0, 150);
79          noFill();
80          // Zeichne einen Kreisbogen exakt im aktiven Arbeitsfenster (30° bis 150°)
81          // Processing nutzt Radian: 0° ist rechts, mathematisch negativ drehend
82          arc(0, 0, r*2, r*2, radians(30 - 180), radians(150 - 180));
83
84          // Distanz-Beschriftung an den Kreisringen platzieren
85          fill(0, 180, 0);
86          textSize(12);
87          int cmText = int(map(r, 0, radarRadius, 0, maxDistanceCm));
88          text(cmText + " cm", 10, -r + 5);
89      }
90
91      // =====
92      // SCHRITT 2: WINKEL-LINIEN & GRAD-BESCHRIFTUNGEN
93      // =====
94      int[] winkelCheck = {30, 60, 90, 120, 150};
95      for (int w : winkelCheck) {
96          stroke(0, 80, 0, 100);
97          float rad = radians(w - 180);
98          float lx = radarRadius * cos(rad);
99          float ly = radarRadius * sin(rad);
100         line(0, 0, lx, ly);
101
102         // Gradzahlen leicht auSSerhalb des äusseren Rings platzieren
103         float tx = (radarRadius + 25) * cos(rad);
104         float ty = (radarRadius + 25) * sin(rad);
105         fill(0, 230, 0);
106         textAlign(CENTER, CENTER);
107         text(w + "°", tx, ty);
108     }
109     textAlign(LEFT, BASELINE); // Textausrichtung für nachfolgende Elemente zurücksetzen
110
111     // =====
112     // SCHRITT 3: INTELLIGENTE KANTENERKENNUNG (OBJEKT-COMPILER)
113     // =====
114     boolean inObject = false;    ///< Flag, ob der Scan-Algorithmus sich gerade innerhalb eines Objekts befindet
115     int startAngle = 0;          ///< Der Startwinkel des aktuell erfassten Objekt-Clusters.
116     float sumDistance = 0;       ///< Akkumulierte Distanzwerte zur späteren Mittelwertbildung.
117     int countPins = 0;           ///< Anzahl der Messpunkte innerhalb dieses Objekt-Blocks.
118     float lastValidDistance = 0; ///< Distanzwert des vorhergehenden Winkelschritts zur Sprunganalyse.
119
120     for (int i = 30; i <= 150; i++) {
121         float d = radarWerte[i];
122
123         // Prüfen, ob ein physikalisches Objekt im Messbereich liegt
124         if (d > 2 && d <= maxDistanceCm) {
125             if (!inObject) {
126                 // Eintrittsflanke: Ein neues Objekt wurde entdeckt
127                 inObject = true;
128                 startAngle = i;
129                 sumDistance = d;
130                 countPins = 1;
131                 lastValidDistance = d;
132             } else {
133                 /* MATHEMATISCHER FILTER GEGEN SCHALLSTREUUNG:

```

```

134         Wenn sich der Abstand zum vorherigen Gradschritt schlagartig um mehr als 3.5 cm
135         ändert, detektiert der Filter eine Objektkante. Das bedeutet, zwei dicht
136         nebeneinander stehende Objekte werden hier sauber getrennt! */
137         if (abs(d - lastValidDistance) > 3.5) {
138             drawCompensatedObject(startAngle, i - 1, sumDistance / countPins);
139             // Unmittelbarer Neustart des Algorithmus für das nächste Objekt
140             startAngle = i;
141             sumDistance = d;
142             countPins = 1;
143         } else {
144             // Keine Kante -> Der Punkt gehört zur Oberfläche desselben Objekts
145             sumDistance += d;
146             countPins++;
147         }
148         lastValidDistance = d;
149     }
150 } else {
151     // Austrittsflanke: Der Strahl wandert wieder ins Leere
152     if (inObject) {
153         drawCompensatedObject(startAngle, i - 1, sumDistance / countPins);
154         inObject = false;
155     }
156 }
157 }
158 // Letztes aktives Objekt am Gehäuserand (150°) sichern
159 if (inObject) {
160     drawCompensatedObject(startAngle, 150, sumDistance / countPins);
161 }
162
163 // =====
164 // SCHRITT 4: AKTUELLER RADARSTRAHL (ZEIGER)
165 // =====
166 strokeWeight(3);
167 stroke(0, 255, 0, 220); // Leuchtendes Neon-Grün
168 float pointerX = radarRadius * cos(radians(angle - 180));
169 float pointerY = radarRadius * sin(radians(angle - 180));
170 line(0, 0, pointerX, pointerY);
171
172 popMatrix(); // Lokales Koordinatensystem schlieSSen
173
174 // =====
175 // SCHRITT 5: STATUS-DASHBOARD (USER INTERFACE)
176 // =====
177 fill(10, 30, 10, 220);
178 stroke(0, 255, 0, 80);
179 rect(15, 15, 260, 110, 8); // Abgerundete Sci-Fi Box
180
181 fill(0, 255, 0);
182 textSize(15);
183 text("RADAR-STATUS: " + systemState, 30, 40);
184
185 if (systemState.equals("OK")) {
186     text("Winkel: " + int(angle) + "°", 30, 70);
187     text("Entfernung: " + int(distance) + " cm", 30, 100);
188 } else if (systemState.equals("TESTING")) {
189     fill(255, 220, 0); // Warnfarbe Gelb während des Bootens
190     text("Selbsttest läuft...", 30, 70);
191     text("LEDs & Sensor prüfen.", 30, 100);
192 } else {
193     fill(255, 50, 50); // Alarmfarbe Rot bei Störung
194     text("HARDWARE FEHLER!", 30, 70);
195     text(errorMessage, 30, 100);
196 }
197 }
198
199 /**
200  * @brief Berechnet die reale Objektgeometrie und zeichnet das kompensierte Hindernis.
201  * @details Diese Funktion entscheidet anhand der akkumulierten Breite im Gradbereich,
202  * wie das Objekt dargestellt wird. Winzige Echos werden zu einem messerscharfen Kern-Punkt zentriert.

```



```

203 * Echte, breite Objekte werden als flächige Kreisbogen-Struktur abgebildet, um die physikalische
204 * Realität perfekt zu spiegeln.
205 * * @param startA Der mathematische Eintrittswinkel des Objekts.
206 * @param endA Der mathematische Austrittswinkel des Objekts.
207 * @param avgDist Die gemittelte, fehlerbereinigte Distanz des Objekts in Zentimetern.
208 */
209 void drawCompensatedObject(int startA, int endA, float avgDist) {
210     int objectWidthDegrees = endA - startA;
211
212     // Extrem schmales Rauschen (unter 2 Grad Breite) als Fehlsignal blockieren
213     if (objectWidthDegrees < 2) return;
214
215     float centerAngle = startA + (objectWidthDegrees / 2.0);
216     float r = map(avgDist, 0, maxDistanceCm, 0, radarRadius);
217
218     stroke(255, 30, 30, 220); // Gefahrenfarbe Rot für Hindnisse
219     fill(255, 0, 0, 200);
220
221     if (objectWidthDegrees <= 7) {
222         // Schmäler Modus: Objekt wird als fokussierter Zielpunkt komprimiert
223         float x = r * cos(radians(centerAngle - 180));
224         float y = r * sin(radians(centerAngle - 180));
225         ellipse(x, y, 7, 7);
226     } else {
227         // Breiter Modus: Objekt wird als reale, flächige Kontur gezeichnet
228         strokeWeight(4);
229         noFill();
230         arc(0, 0, r*2, r*2, radians(startA - 180), radians(endA - 180));
231         strokeWeight(1); // Standard-Dicke wiederherstellen
232     }
233
234     // Eine feine Peillinie zieht die Vektor-Richtung zum Objektzentrum
235     float cx = r * cos(radians(centerAngle - 180));
236     float cy = r * sin(radians(centerAngle - 180));
237     stroke(255, 0, 0, 40);
238     line(cx, cy, 0, 0);
239 }
240
241 /**
242 * @brief Asynchroner Interrupt-Handler für eingehende serielle Daten.
243 * @details Diese Funktion wird von Processing automatisch im Hintergrund getriggert, sobald
244 * ein Zeilenumbruch ('\n') im Puffer des USB-Ports registriert wird. Sie parst das proprietäre
245 * Übertragungsprotokoll des Arduinos:
246 * - "S:[STATUS]" -> Schaltet den globalen Systemzustand um.
247 * - "D:[WINKEL],[DISTANZ]" -> Trennt Winkel und Distanz und speichert sie im globalen Array.
248 * * @param p Das Objekt des seriellen Ports, welcher das Event ausgelöst hat.
249 */
250 void serialEvent(Serial p) {
251     try {
252         String inString = p.readStringUntil('\n');
253         if (inString != null) {
254             inString = trim(inString); // Steuerzeichen und Leerzeichen abschneiden
255
256             // Parser-Zweig 1: Systemmeldungen extrahieren
257             if (inString.startsWith("S:")) {
258                 systemState = inString.substring(2);
259                 if (systemState.equals("ERR_SENSOR")) {
260                     errorMessage = "Sensor-Echo_fehlt!";
261                 }
262             }
263             // Parser-Zweig 2: Numerische Messdaten dekodieren
264             else if (inString.startsWith("D:") && systemState.equals("OK")) {
265                 String[] data = split(inString.substring(2), ',');
266                 if (data.length == 2) {
267                     angle = float(data[0]);
268                     distance = float(data[1]);
269
270                     // Datenbereich absichern und im Ringpuffer-Array hinterlegen
271                     int index = int(angle);

```

```
272         if (index >= 0 && index <= 180) {
273             radarWerte[index] = distance;
274         }
275     }
276 }
277 }
278 } catch (Exception e) {
279     println("Fehler beim seriellen Parsing: " + e.getMessage());
280 }
281 }
```

Teil V.

Developmenmt

13. Flow Chart Development XY

Teil VI.

Monitoring

14. Monitoring XY

Teil VII.

**Verification - Evaluation -
Conclusion**

15. Evaluation/Verification XY

16. To DoX Y

17. Conclusion XY

Teil VIII.

Anhang

18. Bill Of Material

Tabelle 18.1.: Hardware Bill of Materials

Bild	Stückzahl	Beschreibung	Zulieferer	Preis()
	1	Arduino Nano 33 BLE Sense	reichelt.de	33.03
	1	Ultraschallsensor HC-SR-04	reichelt.de	2.90
	1	TowerPro Servomotor SG90	az-delivery.de	5.99
	1	Breadbord 25 Kontakte	reichelt.de	2.39
	1	LED grün 8mm	reichelt.de	0.19
	1	LED rot 8mm	reichelt.de	0.19
	1	Entwicklerboards - Steckbrückenkabel 40 Pole	reichelt.de	1.55
	1	Micro-USB Kabel	reichelt.de	5.26
	2	220 Ohm Widerstand	reichelt.de	0.18
	1	2k Ohm Widerstand	reichelt.de	0.10
	1	1k Ohm Widerstand	reichelt.de	0.09
Total				≈ 51,87

Stichwortverzeichnis Keywords

19. **SBOM**

`requirements.txt`

20. Package Example

20.1. Introduction

20.2. Description

20.3. Installation

`pip packageExample`

20.4. Example - Manual

20.5. Example

20.6. Example - Code

20.7. Example - Files

`PackageExample.py`

20.8. Further Reading

Literatur

- [Aba+16] M. Abadi u. a. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, S. 265–283. ISBN: 978-1-931971-33-1. URL: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.

Teil IX.

**Hints - Examples for LaTeX -
Read, Use and Remove**

21. Hinweise

Bitte verwenden Sie **biber.exe**.

Wenn Sie ein Symbolverzeichnis erstellen möchten, rufen Sie makeindex auf:
makeindex %.nlo -s nomencl.ist

21.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier-Kurven werden mit Hilfe von Bernsteinpolynomen gebildet. Wenn mindestens zwei Punkte sowie zwei Tangenten bekannt sind, können Kontrollpunkte ermittelt werden mit denen ein Kontrollpolygon gebildet wird. An diesem Kontrollpolygon orientiert sich der Verlauf der Kurve. Die Anzahl der Kontrollpunkte ist abhängig vom Grad der Bézier-Kurve. Eine Bézier-Kurve vom Grad n hat $n+1$ Kontrollpunkte. Die Berechnung der Bézier-Kurve erfolgt über den De-Casteljau-Algorithmus.

Definition. Im Intervall $[0; 1]$ ist das **Bernstein-Polynom n -ten Grades** definiert durch:

$$b_{i,n}(\lambda) = \binom{n}{i} (1-\lambda)^{n-i} \lambda^i, \quad \lambda \in [0, 1], \quad i = 0, \dots, n \quad (21.1)$$

Definition. Der Binomialkoeffizient ist definiert durch:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \dots, n \quad (21.2)$$

Hierbei bilden die Bernstein-Polynome $b_{i,n}$ eine Basis des Vektorraumes $\mathbb{P}^n(I)$ der Polynome höchstens vom Grad n über I . Somit lässt sich jedes Polynom höchstens n -ten Grades eindeutig als Linearkombination schreiben. [Far02]

Beispiel. Bestimmung von Bernstein-Polynome für $n = 3$ gilt:

$$\begin{aligned} b_{3,0}(\lambda) &= \binom{3}{0} (1-\lambda)^{3-0} \lambda^0 = (1-\lambda)^3 \\ b_{3,1}(\lambda) &= \binom{3}{1} (1-\lambda)^{3-1} \lambda^1 = 3\lambda(1-\lambda)^2 \\ b_{3,2}(\lambda) &= \binom{3}{2} (1-\lambda)^{3-2} \lambda^2 = 3\lambda^2(1-\lambda) \\ b_{3,3}(\lambda) &= \binom{3}{3} (1-\lambda)^{3-3} \lambda^3 = \lambda^3 \end{aligned}$$

$b_{3,i}(\lambda)$ mit $i = 0, 1, 2, 3$ sind die kubischen Bernstein-Polynome von Grad 3.

Definition. Gegeben seien die Punkte

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \dots, n$$

Eine **Bézier-Kurve** ist dann definiert durch

$$C(\lambda) = \sum_{i=0}^n Q_i \cdot b_{i,n}(\lambda) \quad (21.3)$$

Die Punkte $Q_i, i = 0, \dots, n$ heißen **Kontrollpunkte**.

Die Kontrollpunkte einer Bézier-Kurve bilden das sogenannte Kontrollpolygon.

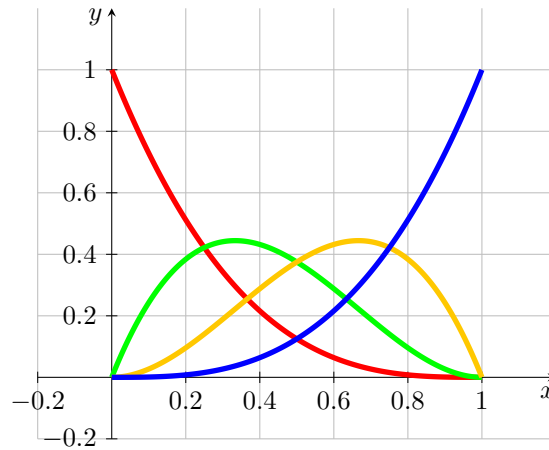


Abbildung 21.1.: Bernstein-Polynom vom Grad 3

Bemerkung. Es sei der Startpunkt P_0 und der Endpunkt P_1 , sowie die Tangenten $\vec{t}_0$ und $\vec{t}_1$ gegeben. Die Tangenten sind nicht notwendigerweise normiert. Die Kontrollpunkte Q_0, Q_1, Q_2 und Q_3 der zugehörigen Bézier-Kurve sind durch folgende Gleichungen gegeben:

$$Q_0 = P_0, \quad Q_1 = P_0 + \lambda_0 \vec{t}_0, \quad Q_2 = P_1 - \lambda_1 \vec{t}_n, \quad Q_3 = P_1 \quad (21.4)$$

[Jak+10]

Beispiel. Gegeben seien

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t}_1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Dann ergibt sich gemäß Satz 21.4 für Kontrollpunkte der zugehörigen Bézier-Kurve:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad Q_1 = P_0 + \vec{t}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t}_n = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \quad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

Die Bézier-Kurve und ihre Kontrollpunkte sind in der Abbildung 21.2 dargestellt.

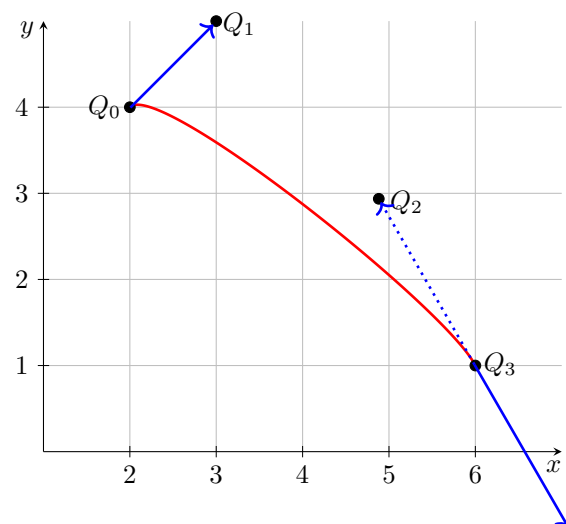


Abbildung 21.2.: Bézier-Kurve zum Beispiel 21.1

22. Verrundung mit einem Kreisbogen

22.1. Gleichungen

Die Verrundung mit einem Kreisbogen ist eine einfache Variante der Glättung von Ecken. Diese Variante ermöglicht die Bearbeitung und die Erzeugung von Dateien gemäss DIN 66025. Zur Glättung werden stets drei Punkte P_0 und S und P_1 betrachtet, damit ergibt sich ein symmetrisches Hermite-Problem. Gemäss Satz ?? wird vorausgesetzt, dass es sich in der Standardform $HP(L, \alpha)$ befindet.

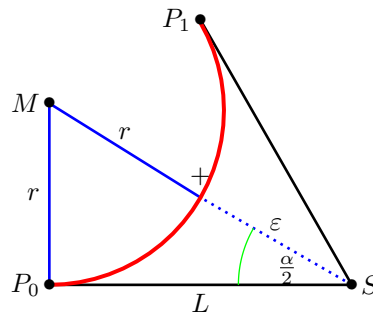


Abbildung 22.1.: Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck

Die Abbildung 22.1 stellt die Situation dar. Die Punkte P_0 , S und P_1 sind gegeben. Der eingeschlossene Winkel $\alpha = \angle(P_0; S; P_1)$ sowie der Abstand $L = \|S - P_0\| = \|P_1 - S\|$ sind in der Grafik dargestellt. Der rote Kreisbogen ist das gewünschte Ergebnis. Der Abstand seines Mittelpunkts M zu S beträgt dann $r + \varepsilon$, wobei r der Radius des Kreisbogens und ε die vorgegebene Toleranz ist. Somit erhalten wir ein rechtwinkliges Dreieck P_0SM , dessen Kantenlängen L , $r + \varepsilon$ und r sind.

Gemäss der Definition des Sinus und des Tangens folgt:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r + \varepsilon} \quad \text{und} \quad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \quad \text{und} \quad r = \cot\left(\frac{\alpha}{2}\right) L$$

Die beiden Gleichung können zusammengefasst werden, so dass sich folgende Bedingungen ergeben:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \quad \text{bzw.} \quad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

Damit ergibt sich für den Radius die Gleichung:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1 - \cos(\alpha)}{1 + \cos(\alpha)}} = L \cdot \frac{\sin(\alpha)}{1 + \cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

Der Faktor zum Umrechnung von L und ε kann mit Hilfe trigonometrischer Umformungen vereinfacht werden.

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1 - \sqrt{\frac{1 - \cos(\alpha)}{2}}}{\sqrt{\frac{1 + \cos(\alpha)}{2}}} = \frac{\sqrt{2} - \sqrt{1 - \cos(\alpha)}}{\sqrt{1 + \cos(\alpha)}} = \frac{\sqrt{1 + \cos(\alpha)} - \sin(\alpha)}{1 + \cos(\alpha)}$$

Durch den Vergleich mit dem symmetrischen Hermite-Problem in Standardform ergibt sich dann die folgende Notation:

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}}$$

Die vorherigen Überlegungen werden nun im nachfolgenden Satz zusammengefasst.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

Dann kann ein Kreisbogen gefunden werden, der die Punkte P_0 und P_1 verbindet, die gleichen Tangentenrichtungen in den beiden Punkten besitzt.

Für die Kreisbogen gilt:

$$r = L \cdot \left| \frac{P_{1,x}}{P_{1,y}} \right|; \quad \phi_0 = -\text{sign}(\alpha) \cdot \frac{\pi}{2}; \quad \phi_1 - \phi_0 = \text{sign}(\alpha) \cdot \pi - \alpha = \beta;$$

$$M = P_0 + \text{sign}(\alpha) \cdot r \cdot \vec{t}_0^\perp = \text{sign}(\alpha) \cdot r \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Aus der Vorgabe des Abstand L kann, wie in Satz 22.1 dargestellt, der maximale Fehler berechnet werden. Gemäss der Herleitung ist es auch möglich, den maximalen Fehler ε vorzugeben und daraus den maximalen Abstand L zu bestimmen.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

- a) Es sei der maximale Fehler ε vorgegeben. Der maximale Abstand L , für den ein Kreisbogen gemäss Satz 22.1 existiert, der den Fehler berücksichtigt, ist gegeben durch:

$$L(\varepsilon, \alpha) = \varepsilon \cdot \frac{P_{1,x}}{\sqrt{P_{1,x}} - P_{1,y}} = \varepsilon \cdot \frac{L + L \cdot \cos(\alpha)}{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}$$

- b) Bei der Vorgabe des Abstands L ergibt sich der folgende, maximale Fehler:

$$\varepsilon(L, \alpha) = L \cdot \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}} = L \cdot \frac{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}{L + L \cdot \cos(\alpha)}$$

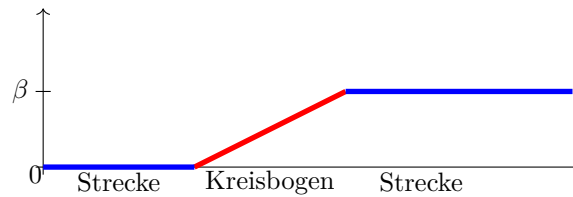


Abbildung 22.2.: Winkeländerung bei einer Verrundung mit einem Kreisbogen

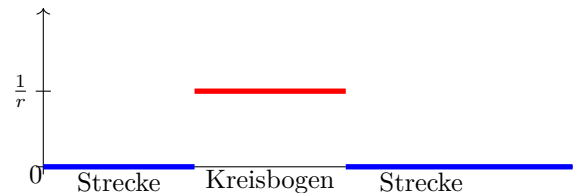


Abbildung 22.3.: Krümmungsverlauf bei der Verrundung mit einem Kreisbogen

Aus diesen Überlegungen ergeben sich Einschränkungen für den Einsatz dieser Strategie, die in der folgenden Bemerkung zusammengefasst sind.

Bemerkung. Folgende Bedingungen zum Anwenden der Strategie eines Kreises müssen erfüllt sein:

a)

$$P_0 \neq P_1$$

b)

$$\vec{t}_0 = -\vec{t}_1 \Leftrightarrow \alpha = 0$$

c)

$$\vec{t}_0 = \vec{t}_1 \Leftrightarrow \alpha = \pi$$

22.2. Bewertung

Die Verrundung mittels eines Kreisbogens führt zu einem stetigen Verlauf der Winkeländerung. Die Abbildung 22.2 stellt dies dar.

Durch die Verrundung mit einem Kreisbogen wird ist der Krümmungsverlauf der Kurve nicht stetig. Die Abbildung 22.3 zeigt, dass die Krümmung im Bereich der Strecken 0 ist und auf dem Kreisbogen konstant mit dem Wert $\frac{1}{r}$, wobei r der Radius des eingesetzten Kreisbogens ist. Aufgrund der Formel $a = \frac{v^2}{r}$ für eine Bewegung auf einem Kreisbogen weist der Beschleunigungsverlauf bei einer konstanten Bahngeschwindigkeit Stetigkeitssprüngen an den Übergängen aus.

In Abhängigkeit der Winkeländerung β an der Ecke S und der Toleranz ε kann die maximale Länge der Verkürzung der Strecken berechnet werden. Die Abbildung 22.4 zeigt das zugehörige Diagramm, wobei die Toleranz ε auf den Wert 1 gesetzt ist.

Der Bereich, der zur Verfügung gestellt wird, kann auch vorgegeben werden. In Abhängigkeit des Abstands L vom Eckpunkt S kann die Abweichung ε berechnet

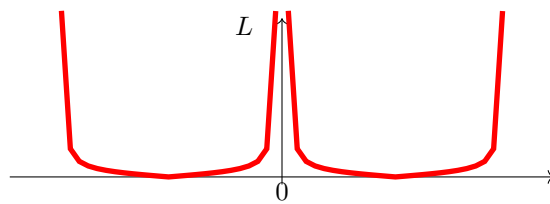


Abbildung 22.4.: Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = \text{const}, \alpha)$

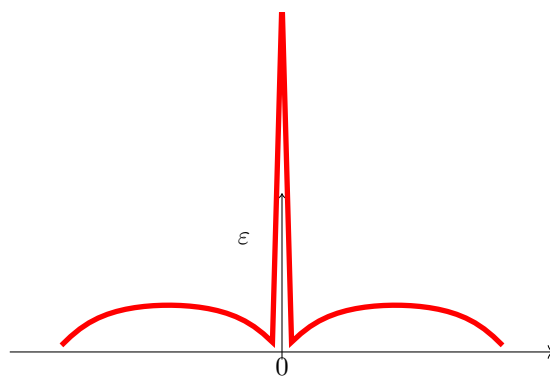


Abbildung 22.5.: Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen

werden. Die Abbildung 22.5 stellt die Funktion in Abhängigkeit von L und des Winkels α dar.

Die Abbildungen 22.4 und 22.5 zeigen die Kurven der Länge in Abhängigkeit der Winkeländerung bei einer festen Toleranz und den Fehler in Abhängigkeit der Winkeländerung bei vorgegebenen Länge. Falls die Winkeländerung minimal ist, dann ist der Fehler oder die Länge L extrem. In diesem Fall ist eine Verrundung mittels eines Kreisbogens nicht möglich. Um eine sinnvolle Strategie zu entwickeln, muss eine minimale Winkeländerung vorgegeben werden, ab der eine Verrundung mit einem Kreisbogen erlaubt ist. Ebenso ist eine maximale Winkeländerung zu definieren. Denn bei einer Winkeländerung von $\pm\pi$ handelt sich um eine Kehre. in diesem Fall ist eine Verrundung auch nicht möglich.

22.3. Beispiele

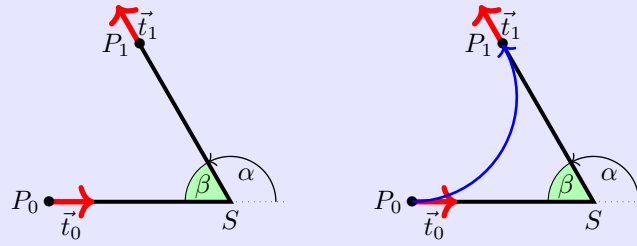
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{2}{3}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$



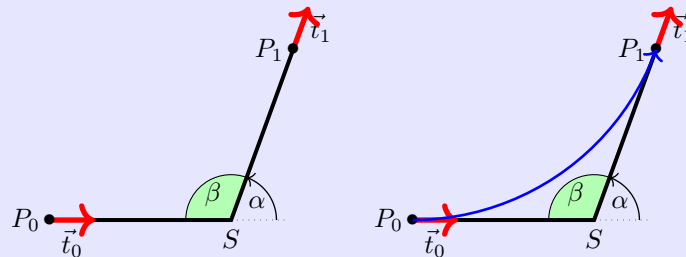
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{7}{18}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$



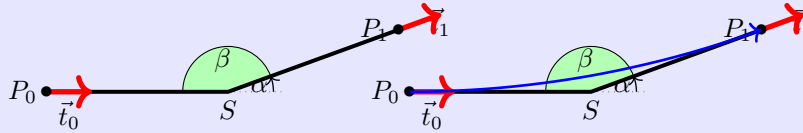
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{1}{9}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

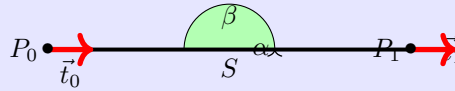


Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = 0$ gegeben.

Hier existiert kein Verrundungskreisbogen.



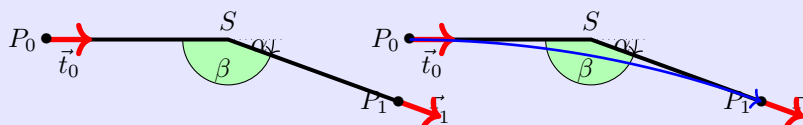
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{1}{9}\pi\right) \\ \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{1}{9}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{1}{9}\pi$$



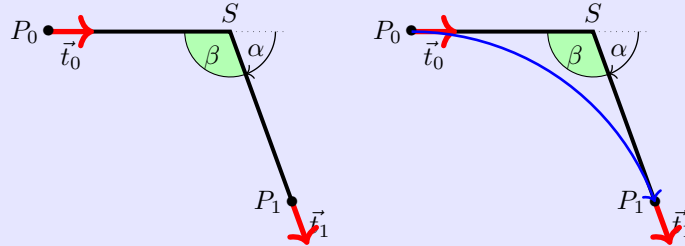
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{7}{18}\pi\right) \\ \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{7}{18}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$



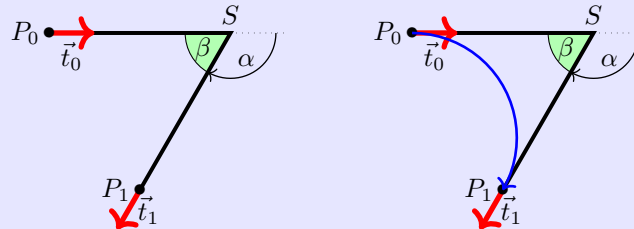
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{2}{3}\pi\right) \\ \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{2}{3}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -3,4641 \end{pmatrix}; \quad r = 3,4641; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



23. Maple-Dateien

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

23.1. Geometrien mit Maple

Die vorgestellten Algorithmen können gut mit Hilfe von Geometrien dargestellt werden. Dabei können zwei Aspekte untersucht werden. Einerseits ist der Aufwand für eine Umsetzung, die Rechengenauigkeit und die Stabilität eines Algorithmus interessant; andererseits sollen die Verfahren hinsichtlich ihres Nutzens bewertet und verglichen werden. Hierzu wurden Module für das Formelmanipulationssystem Maple [Wat17c] entwickelt.

Maple bietet die Umgebung, Algorithmen einfach und schnell zu implementieren. Darüber hinaus ist die grafische Darstellung mit einfachen Mitteln möglich. Allerdings ist eine einfache Arbeitsmappe von Maple nicht für sehr große Software-Projekte ausgelegt. Hier muss auf die Möglichkeit der Erstellung und Nutzung von Bibliotheken zurückgegriffen werden. Einerseits bietet Maple die Möglichkeit, eigene Bibliotheken, so genannte Module, zu erstellen. Auf diese Weise bleibt man innerhalb der Umgebung und der Syntax von Maple. Dieser Weg wird hier weiter verfolgt. Eine weitere Möglichkeit ist die Verwendung von Bibliotheken, die mittels einer höheren Programmiersprache, z.B. C++, erstellt wurde, den so genannten DLLs.

Im weiteren wird die Erstellung und Verwendung einer Testumgebung mit Hilfe von Module beschrieben. Zunächst werden die Geometrieelemente beschrieben, die in verschiedenen Module abgelegt sind, und deren Verwendung. Damit eigene Erweiterungen und Ergänzungen möglich sind, wird dann der Aufbau und die Verwendung eines Moduls in Maple erläutert.

23.2. Verwendete Geometrieelemente

Da es sich um eine Testumgebung für die vorgestellten Algorithmen, werden auch nur Geometrien verwendet, die beschrieben wurden.

- Punkte (**MPoint**)
- Strecken (**MLine**)
- Kreisbögen (**MArc**)
- Bézier-Kurven (**Bezier**)
- Polygonzüge (**MPolygon**)
- Geometrieliste (**MGeoList**)
- Hermite-Probleme (**MHermiteProblem**)
- Symmetrische Hermite-Probleme (**MHermiteProblemSym**)

Für jedes Geometrieelement wurde ein entsprechendes Modul angelegt, dessen Name in der obigen Liste angegeben ist. Die Auflistung, welche Funktionen zur Verfügung stehen, wird in einem weiteren Abschnitt dargestellt.

Bei der Umsetzung eines solchen Projekts wird schnell deutlich, dass bei einer Verwendung von mehreren Geometrieelementen eine klare Datenstruktur und ein wohldefinierter Zugriff auf die Daten unerlässlich ist. Zum Beispiel bei der Darstellung von Geraden ist die Entscheidung zu treffen, ob die Darstellung Punkt-Richtung oder mittels zweier Punkte erfolgen soll. Die Auswahl erfolgt im Allgemeinen in Abhängigkeit des Anwendungsfalls. Hier wird der Weg beschritten, dass aufgrund eines wohldefinierten Zugriffs auf die Daten die Verwendung beider Darstellungen möglich ist.

23.3. Aufbau der Datenstruktur

Die Idee ist, eine möglichst einheitliche und einfache Datenstruktur zu verwenden. Maple bietet zwar die Möglichkeit, Objekte zu definieren. Allerdings ist die Verwendung innerhalb einer prozeduralen Umgebung schwierig. Daher werden alle Daten grundsätzlich als Listen dargestellt. Das erste Listenelement enthält dabei stets eine Kennung des Elements `??`. Dann folgen die Daten, die wiederum Geometrieelemente sein können. Es ist zu beachten, dass der direkte Zugriff auf die Daten nicht unterbunden werden kann; hier ist der Anwender in der Pflicht.

Der Zugriff auf die Daten soll ausschließlich über Prozeduren eines Moduls erfolgen. Im folgenden ist beispielhaft die Datenstruktur für Punkte zu sehen:

```
[MVPOINT, [x, y]]
```

Die Erstellung eines Punkts erfolgt dann über eine Prozedur `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

Beim Aufruf in der Testdatei wird dann folgendes ausgegeben:

```
TestP0 := [PPoint", [10,15]]
```

Diese Datenstrukturen werden für die Berechnung der Geometrien verwendet. Mit ihnen wird in Funktionen bzw. Prozeduren gearbeitet. Anders als Variablen werden die Datenstrukturen und Prozeduren hier global und nicht lokal definiert. Unter dem Befehl `export` werden die Prozeduren zu Beginn des Moduls deklariert und sind so auch außerhalb des Moduls verwendbar. Soll beispielsweise eine Datenstruktur aus `MPoint` in einem anderen Modul oder außerhalb der Module verwendet werden, wird sie wie folgt aufgerufen:

```
P0 := MPoint:-New(x,y);
```

Zusätzlich zu den Modulen für die Geometrien gibt es ein Modul (`MConstant`) zum Speichern von Konstanten und Namen. Dort ist für `MVPOINT` z.B. „Point“ gespeichert oder z.B. eine Konstante für den Vergleich auf Null für reelle Zahlen.

Zuletzt gibt es die Testdatei, welche kein Modul ist, in der die Module geladen, aufgerufen und in ihrer Funktion getestet werden. Zu dieser Datei später im Punkt „1.4 Maple Testdatei“ mehr.

23.4. Struktur eines Moduls

In dem folgenden Abschnitt geht es um den Zweck von Modulen und deren Struktur eingegangen.

Es geht in dem Projekt darum die oben genannten Geometrien in einer Datenstruktur zu erfassen und sie in Maple darzustellen. Für die letztendliche Darstellung sind die Berechnungen und Formeln, die verwendet werden müssen, allerdings hinderlich. Module eignen sich sehr gut dafür die Berechnungen, die in Funktionen erfolgen,

zusammenzufassen und zu verstecken. So kann in Maple auf die wichtigen Funktionen zugegriffen werden, ohne dass man sieht, was in diesen steht.

Beispiel:

Die Funktion **Angle** aus dem Modul **MPoint**: Funktion zur Berechnung eines Winkels zwischen Ortsvektor und x-Achse:

```
Angle := proc(P)
    local alpha, x, y;
    x := GetX(P);
    y := GetY(P);
    alpha := 0;
    if abs(x) < 0.00001
    then
        alpha := 3*Pi/2;
    else
        alpha := Pi/2;
    end if;
else
    alpha := arctan(y/x);
    if x < 0
    then
        alpha := alpha + Pi;
    else
        if y < 0
        then
            alpha := alpha + 2*Pi;
        end if;
    end if;
    end if;
    return factor(alpha);
end proc;
```

Diese Funktion ist lang, stellt aber nur einen kleinen Teil des Programms für die Darstellung dar, um die es geht. Deshalb steht sie in dem Modul und kann so extern mit einem einzigen Befehl aufgerufen werden:

```
Winkel := MPoint:-Angle(P)
```

Im Folgenden Abschnitt wird die Struktur eines Moduls beschrieben. Da Maple hier sehr vielfältige Möglichkeiten bietet, findet eine Beschränkung statt. Es wird nur die Struktur der verwendeten Module, hier anhand des Moduls **MPoint**, beschrieben. Für tiefergehende Möglichkeiten wird hier auf das Handbuch von Maple [Wat17c] verwiesen.

Struktur:

Das Modul muss zunächst gestartet werden. Dies erfolgt mit dem Namen (hier immer ein groSSes M und die Geometrie) des Moduls und dem folgenden Befehl:

```
MPoint := module()
```

Danach müssen, ähnlich wie in Prozeduren, die Variablen und Funktionen definiert werden. Dabei kann man entweder lokal oder global deklarieren. Werden die Variablen

oder Prozeduren nur in dem Modul gebraucht und verändert, erfolgt die Deklaration mit dem Befehl `local` wie folgt:

```
local Variablenamen, mit, Komma, getrennt;
```

Sollen die Variablen oder Prozeduren auch auSSerhalb des Moduls verwendbar sein, wird mit `export` dies definiert:

```
export Funktionsnamen, mit, Komma, getrennt;
```

Daraufhin folgt die Angabe der Optionen:

```
option package;
```

die Beschreibung des Moduls:

```
description "Selbstgewählte Modulbeschreibung z.B. Modul für Punkte";
```

und die Initialisierung des Moduls:

```
ModuleLoad := proc
  MVPPOINT:=MConstant:-GetPoint();
  print("Modul MPoint ist geladen");
end proc;
```

Ab hier wird programmiert wie sonst in Maple auch. Man verwendet die vorher definierten Prozedur- und Variablenamen und programmiert mit Befehlen die man auSSerhalb eines Moduls in Maple auch verwendet. Wichtig zu wissen ist, dass bei Modulen jede Funktion ständig abgerufen werden kann, die Reihenfolge der Funktionen also keine Rolle spielt.

Um zuletzt das Modul zu beenden, nutzt man den folgenden Befehl:

```
end module;
```

23.4.1. Genereller Aufbau eines Moduls

Zusammenfassend haben die Module also folgendes Gerüst:

```
Modulname := module()

  local Namen, mit, Komma, getrennt;

  export Namen, mit, Komma, getrennt;

  global Namen, mit, Komma, getrennt;1

  option package;

  description "Selbstgewählte Modulbeschreibung";

  ModuleLoad := proc()2
    MVPPOINT:=MConstant:-GetPoint();
    print("Modul MPoint ist geladen");
  end proc;

  Procedure1 := proc (Übergabeparameter)
```

¹möglich, aber in den Modulen nicht verwendet

²Beispiel `MPoint`

```
        inhalt;  
    end proc;  
  
    Procedure2 := proc (Übergabeparameter)  
        inhalt;  
    end proc;  
  
    ...  
end module;
```

23.4.2. Sicherung eines Moduls

Die Verwaltung von Module wird von Maple automatisch durchgeführt. Da aber die Module weitergegeben werden und einzeln zu bearbeiten sind, müssen einige Einstellungen vorgenommen werden. Daher wird bei jeder Maple-Datei des Projekts folgender Präfix verwendet:

```
1 restart;
2 with(LibraryTools);
3 lib := C:/FH/Tools/Maple/MyLibs/Blending.mla";
4 march('open', lib);
5 ThisModule := 'MArc';
```

Die 1. Zeile initialisiert das System. Die nächsten 3 Zeile ermöglichen das Arbeiten mit Archiven. In der zweiten Zeile werden die Werkzeuge geladen, so dass die Variable **lib** belegt werden kann. Alle Module werden in einem Archiv abgelegt; in diesem Beispiel ist es die Datei **Blending.mla** im Verzeichnis **C:/FH/Tools/Maple/MyLibs/**. Das Kommando **ThisModule := 'MArc';** enthält den Namen des aktuellen Moduls.

Ein Modul wird dann mittels des Befehls **savelib('ModuleName')** gespeichert. Es wird dann eine Datei **ModuleName.mla** angelegt. Je nach Konfiguration von Maple ist der eingestellte Pfad, wo die Datei automatisch angelegt wird, nicht beschreibbar. Dann kann man das System so konfigurieren, dass die mla-Datei im aktuellen Verzeichnis oder in einem Verzeichnis der eigenen Wahl abgelegt wird.

Falls der obige Vorspann für eine Maple-Datei verwendet wird, genügt zum Speichern des Moduls

```
savelib(ThisModule, lib);
```

23.4.3. Verwendung eines Moduls

Ein Modul, das abgespeichert wurde, kann nun in anderen Maple-Worksheets verwendet werden. Dazu wird der Befehl

```
with(ModuleName)
```

verwendet. Falls man einen speziellen Pfad verwendet hat, so kann Maple die Datei **ModuleName.mla** nicht finden. Dann muss der Pfad bekannt gemacht werden, z.B.:

```
savelibname := c:/Maple/MyLibs";", savelibname;
```

Nun kann man auf die Prozeduren des Moduls zugegriffen werden. Eine Übersicht über alle exportierten Prozeduren wird durch den Befehl

```
Describe(ModuleName)
```

angezeigt. Dabei wird eine List der Namen inklusiver der Namen der Übergabeparameter dargestellt. Falls eine Prozedur mit der Feld **description** ausgestattet ist, so wird dieser Text zusätzlich wiedergegeben.

23.4.4. Erstellung eines Maple-Moduls

Die Erstellung eines eigenen Moduls ist schnellst erledigt, falls man den obigen Rahmen verwendet. Allerdings sollte man vorher einige Überlegungen anstellen und einen Standard pflegen.

Bevor ein eigenes Modul erstellt wird, sollte das Datenmodell und die zugehörigen Prozeduren erarbeitet sein. Ein Programmablaufplan leistet hier gute Dienste. Die zentrale Aufgabe des Moduls ist in der Regel schnell ermittelt. Zusätzlich sollten aber folgende Regeln eingehalten werden.

Regel 1 Grundsätzlich erhält sowohl das Modul als auch jede Prozedur eine Beschreibung. Diese beschränkt sich nicht auf das optionale Argument **description**,

sonder wird jeder Prozedur vorangestellt. Die Beschreibung enthält grundsätzlich die Beschreibung der Aufgabe. Dabei werden auch die Voraussetzung genannt bzw. eine Fehlerbehandlung beschrieben. Dann folgt die Beschreibung aller Eingabeparameter, deren Funktion und deren Datenstruktur. Der Rückgabewert wird anschließend beschrieben.

Regel 2 Jedes Modul erhält eine Prozedur `Version()`, die die aktuelle Versionsnummer zurückliefert.

Regel 3 Daten sind nicht global. Um auf Daten zugreifen zu können, werden Prozeduren `Set*` und `Get*` zur Verfügung gestellt. Auch innerhalb der Prozeduren eines Moduls werden diese Prozeduren verwendet.

Regel 4 Für jede Prozedur wird mindestens eine Testfunktion geschrieben. Anhand der Testfunktion kann die Verwendung und ggfs. Besonderheiten dargestellt werden.

23.5. Funktionen der Module

Im folgendem werden die einzelnen Module aufgeführt. Die Datenstruktur jedes Moduls und alle Funktionen mit zugehöriger Aufgabe werden genannt.

23.5.1. Modul `MPoint`

`MPoint` ist ein Modul für Punkte und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur `New` für einen Punkt ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOINT, [x,y]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name `PPoint` und die Elemente sind die x- und die y-Koordinate für einen Punkt. Es wird hier kein Unterschied zwischen Punkt und Vektor gemacht. Ein Punkt kann auch als Vektor vom Koordinatenursprung zum Punkt gesehen werden.

Über die Get-Funktionen `GetX` und `GetY` kann man die Koordinaten des Punktes erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten/Vektoren dann gerechnet werden.

MPoint

E	New	Datenstruktur für einen Punkt
E	GetX	Lesen der x-Koordinate
E	GetY	Lesen der y-Koordinate
E	Angle	Berechnung des Winkels zwischen x-Achse und dem Punkt
E	Add	Errechnet eine Linearkombination zweier Punkte/Vektoren
E	Sub	Errechnet die Differenz zweier Punkte/Vektoren
E	Cos	Errechnet den Cosinus zwischen zwei Vektoren
E	Sin	Errechnet den Sinus zwischen zwei Vektoren
E	Scale	Skaliert einen Vektor mit einem Faktor
E	Perp	Errechnet einen Vektor, der orthogonal zum angegebenen Vektor ist
L	IllustrateXY	Plot Funktion zur Darstellung eines blauen Punktes
E	Illustrate	Darstellung eines blauen Punktes
E	Plot	Darstellung eines grünen Punktes
E	Plot2D	Darstellung eines Punktes mit eigenen Optionen
E	Length	Errechnet den Abstand des Punktes zum Koordinatenursprung
E	Uniform	Normiert den Vektor auf die Länge 1
E	LinetoVector	Errechnet Vektor aus einer Strecke
E	Distance	Errechnet den Abstand zwischen zwei Punkten

23.5.2. Modul MLine

MLine ist ein Modul für Strecken und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

```
[MVLIN, [P0,P1]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Line“ und die Elemente sind der Start- und der Endpunkt für eine Strecke. Die Datenstruktur **NewPointVector** ist ebenfalls eine Datenstruktur für eine Strecke, allerdings wird die Strecke hier über einen Startpunkt und einen Richtungsvektor definiert.

Über die Get-Funktionen **StartPoint** und **EndPoint** kann man Start- und Endpunkt der Strecke erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Strecken dann gerechnet werden.

MLine

E	New	Datenstruktur für eine Strecke (Zwei-Punkt-Form)
E	NewPointVector	Schaffung einer Strecke (Punkt-Richtungs-Form)
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	Position	Errechnen eines Punktes der auf der Strecke liegt
E	Plot2D	Darstellung eines Teils der Strecke ausgehend vom Startpunkt
E	Plot2DTangent	Darstellung eines Punktes mit Tangente
E	Plot2DTangentArrow	Darstellung eines Punktes mit Tangente (als Pfeil)
E	LineLine	Errechnung des Schnittpunktes zweier Geraden
E	AngleLineLine	Errechnung des Winkels zwischen zwei Geraden

23.5.3. Modul **MArc**

MArc ist ein Modul für Kreisbögen und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für einen Kreisbogen ist eine Liste, die wie folgt aufgebaut ist:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name **MArc** und die Elemente sind die x- und y-Koordinate für den Mittelpunkt, der Radius, der Startwinkel und die Winkeländerung für einen Kreisbogen.

Über die Get-Funktionen **GetM**, **GetMX**, **GetMY**, **GetR**, **GetPhi**, und **GetAlpha** kann man die Daten für einen Kreisbogen erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Kreisbögen dann gerechnet werden.

MArc

E	New	Datenstruktur für einen Kreisbogen
E	GetMX	Lesen der x-Koordinate des Mittelpunktes
E	GetMY	Lesen der y-Koordinate des Mittelpunktes
E	GetR	Lesen des Radius
E	GetPhi	Lesen des Startwinkels
E	GetAlpha	Lesen der Winkeländerung
E	GetM	Lesen des Mittelpunktes
E	Position	Errechnung eines Punktes auf dem Kreisbogen
E	Plot2D	Darstellung eines Teils des Kreisbogens ausgehend vom Startwinkel
E	Blend	Errechnung eines Kreisbogens aus einem symmetrischen Hermite-Problem

23.5.4. Modul **MBezier**

Die Datenstruktur **New** für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVBIEZIER, [PointList]]
```

Innerhalb des Moduls **MBezier** existieren die in folgender Tabelle aufgeführten Prozeduren.

MBezier

E	New	Manuelle Eingabe von Kontrollpunkten für eine Bézier-Kurve
E	Version	Ausgabe der Version
E	BlendCurvature	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem
E	BlendCurvatureEpsilon	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem mit vorgegebenen Fehler
E	Position	Position auf der Bézier-Kurve
E	GetTheta	Lesen des Winkels
E	GetEpsilon	Lesen der Toleranz
E	GetControlPoint	Lesen der Kontrollpunkte
E	Plot2D	Darstellung der Bézier-Kurve
E	PlotControlPoints	Darstellung aller Kontrollpunkte

23.5.5. Modul **MPolygon**

MPolygon ist ein Modul für Polygonzüge und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOLYGON, [PointList]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name **PPolygon** und die Elemente sind beliebig viele Punkte.

Über die Get-Funktionen **GetPoint** und **GetN** kann man die Anzahl der Punkte und die Punkte selbst erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten bzw. dem Polygonzug dann gerechnet werden.

MPolygon

E	New	Datenstruktur für eine Punkteliste
E	GetPoint	Lesen des i-ten Punktes aus der Punkteliste
E	GetN	Bestimmung der Anzahl der Punkte in der Punkteliste
E	Length	Bestimmung der euklidischen Länge des Polygonzuges
E	Position	Berechnung eines Punktes auf dem Polygonzug
E	Tangents	Berechnung der Tangenten
L	Plot2DAll	Plotliste aller Punkte
E	Plot2D	Darstellung aller Punkte als Polygonzug
E	Plot2DTangent	Darstellung des Polygonzuges mit Tangenten
E	PlotPoints	Darstellung aller Punkte

23.5.6. Modul **MGeoList**

MGeoList ist ein Modul für eine Geometrieliste und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Geometrieliste ist eine Liste, die wie folgt aufgebaut ist:

```
[MVGEOLIST, []]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „GeoList“ und die Elemente sind beliebig viele einzelne Geometrien.

Über die Get-Funktionen **GeoGeo** und **GetN** kann man das i-te Element der Liste und die Anzahl der Elemente in der Liste erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Geometrien bzw. der Liste dann gerechnet werden. In den Funktionen **Length**, **Plot2DAll**, **Plot2D** und **Position** werden die Funktionen in sich selber aufgerufen. Dies ist möglich da die Funktionen unabhängig voneinander funktionieren.

MGeoList

E	New	Datenstruktur für eine Geometrieliste
E	Append	Anfügen eines Geometrieelements in die Datenstruktur
E	Prepend	Einfügen eines Geometrieelements als erstes Element der Liste
E	Replace	Ersetzen eines Geometrieelements durch ein Anderes
E	GetN	Bestimmung der Anzahl der Geometrieelemente in der Liste
E	GeoGeo	Lesen des i-ten Geometrieelements
E	Length	Errechnet die euklidische Länge der Geometrieelemente
E	Position	Berechnung eines Punktes auf der Geometrie
L	Plot2DAll	Plot Funktion für die Darstellung der Geometrieelemente
E	Plot2D	Darstellung eines Teils der Geometrieliste ausgehend vom ersten Element

23.5.7. Modul MHermiteProblem

MHermiteProblem ist ein Modul für Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „HermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten (Strecken).

Über die Get-Funktionen **StartPoint**, **EndPoint**, **StartTangent** und **EndTangent** kann man die Punkte und ihre Tangenten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das Hermite-Problem dann gerechnet werden.

MHermiteProblem

E	New	Datenstruktur für ein Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	Plot2D	Darstellung des Hermite Problems

23.5.8. Modul MHermiteProblemSym

MHermiteProblemSym ist ein Modul für symmetrische Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für ein symmetrisches Hermite-Problem ist eine Liste, die wie folgt aufgebaut ist:

[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „SymHermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten, ihr Schnittpunkt, und der Abstand der Punkte zum Schnittpunkt.

Über die Get-Funktionen **StartPoint**, **EndPoint**, **StartTangent**, **EndTangent**, **CrossPoint** und **ParameterL** kann man die Daten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das symmetrische Hermite-Problem dann gerechnet werden.

MHermiteProblemSym

E	New	Datenstruktur für ein Symmetrisches Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	ParameterL	Lesen des Abstandes
E	CrossPoint	Lesen des Schnittpunktes
E	Plot2D	Darstellung des symmetrischen Hermite Problems
E	Create	Erzeugung eines Sym. Hermite-Problems durch 3 Punkte
E	BlendArc	Verrundung des Eckpunktes durch einen Kreisbogen

23.5.9. Modul **MConstant**

MConstant ist ein Modul zum Speichern von festen Konstanten und Namen zur Kennung der Datenstrukturen. In der lokal deklarierten Funktionen, beginnend mit CV, werden die Konstanten zur Deklaration der verschiedenen Geometrien definiert. Dies sind Namen zur Erkennung der Geometrieelemente in der Testdatei. In den global deklarierten Funktionen, beginnend mit Get, erfolgt die Rückgabe der Namen zur Kennung der Geometrieelemente.

MConstant

L	NULLEPS	Konstante zum Vergleich auf Null
L	CVPOINT	Konstante für Geometrieelemente: Point
L	CVLINE	Konstante für Geometrieelemente: Line
L	CVARC	Konstante für Geometrieelemente: Arc
L	CVPOLYGON	Konstante für Geometrieelemente: Polygon
L	CVGEOLIST	Konstante für Geometrieelemente: GeoList
L	CVHERMITEPROBLEM	Konstante für Geometrieelemente: HermiteProb
L	CVHERMITEPROBLEMSYMMETRIC	Konst. für Geometrieelemente: SymHermiteProb
L	CVBIARC	Konst. für Geometrieelemente: Biarc
E	GetNullEps	Rückgabe des Nullvergleichs
E	GetPoint	Kennung für Punkte
E	GetLine	Kennung für Strecken
E	GetArc	Kennung für Kreisbögen
E	GetPolygon	Kennung für Polygone
E	GetGeoList	Kennung für Geometrielisten
E	GetHermiteProblemSymmetric	Kennung für symmetrische Hermite-Probleme
E	GetHermiteProblem	Kennung für Hermite-Probleme
E	GetBiarc	Kennung für Biarcs

23.5.10. Modul **MGeneralMath**

MGeneralMath ist ein Modul für allgemeine mathematische Funktionen und arbeitet mit den Datenstrukturen und Prozeduren.

MGeneralMath

E	MPoint	Datenstruktur für einen Punkt
E	MPointX	Lesen der x-Koordinate für einen Punkt
E	MPointY	Lesen der y-Koordinate für einen Punkt
L	MPointIllustrateXY	Plotstruktur für einen blauen Punkt
E	MPointIllustrate	Darstellung eines blauen Punktes
E	MPointPlot	Darstellung eines grünen Punktes
E	MLine	Datenstruktur für eine Strecke
E	MLineStartPoint	Lesen des Startpunktes für eine Strecke
E	MLineEndPoint	Lesen des Endpunktes für eine Strecke
E	MPointOnLine	Berechnung eines Punktes auf der Strecke
E	MLinePlot2D	Darstellung des Teils einer Strecke beginnend beim Startpunkt
E	MLineLine	Errechnen des Schnittpunktes zweier Strecken

23.5.11. Modul Biarc**Datenstruktur**

Voraussetzungen und Festlegungen:

Der Biarc soll ein dafür verwendet werden ein Hermite-Problem zu lösen. Ein Hermite Problem ist wie folgt definiert:

Gegeben seien zwei Punkte P_0 und P_1 mit zugehörigen normierten Tangenten $\vec{t}_0$ und $\vec{t}_1$. Der Biarc muss die Punkte so verbinden, dass die Tangenten des Start- und des Endpunktes des Biarcs mit den Tangenten des Hermites-Problems übereinstimmen.

Datenstruktur:

Die Datenstruktur **New** für einen Biarc muss zwei Kreisbögen enthalten.

Benötigt wird also die Datenstruktur für einen Kreisbogen: **MArc:-New**

Diese enthält wiederum die Koordinaten für den Mittelpunkt, den Radius, den Startwinkel und die Winkeländerung.

23.5.12. Modul MBiarc

MBiarc ist ein Modul für Biarcs und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für einen Biarc ist eine Liste, die wie folgt aufgebaut ist:

[MVBIARC, [Arc0, Arc1]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Biarc“ und die Elemente sind zwei Kreisbögen.

Über die Get-Funktionen **GetArc0** und **GetArc1** kann man die beiden Kreisbögen für den Biarc erfassen. Mit den unten aufgeführten Funktionen kann man die Daten für den Biarc errechnen.

MBiarc

E	New	Datenstruktur für einen Biarc
E	GetArc0	Lesen des ersten Kreisbogens
E	GetArc1	Lesen des zweiten Kreisbogens
E	Circle	Berechnung des Kreises K_J aus den Daten des Hermite-Problems
E	Plot2DCircle	Darstellung des Kreises K_J
E	angle	Errechnet den Winkel vom Kreismittelpunkt zu Punkten auf dem Kreis
E	Plot2D	Darstellung des Biarcs
E	ConnectionPoint	Errechnung des Verbindungspunktes J (Equal Chord)
E	TangentTj	Errechnen der Tangente an J
E	Tangent Biarc	Drehung von Tj für den Biarc
E	BiarcCenter	Errechnen der Mittelpunkte der Kreisbögen des Biarcs
E	Biarc	Errechnen des Biarcs
E	Position	Ermitteln eines Punktes auf dem Biarc
E	Blend	Errechnen des Biarcs nur aus dem Hermite-Problem

23.6. Programmablaufplan

23.6.1. Gesamauf

23.6.2. Verrundung der Kurve

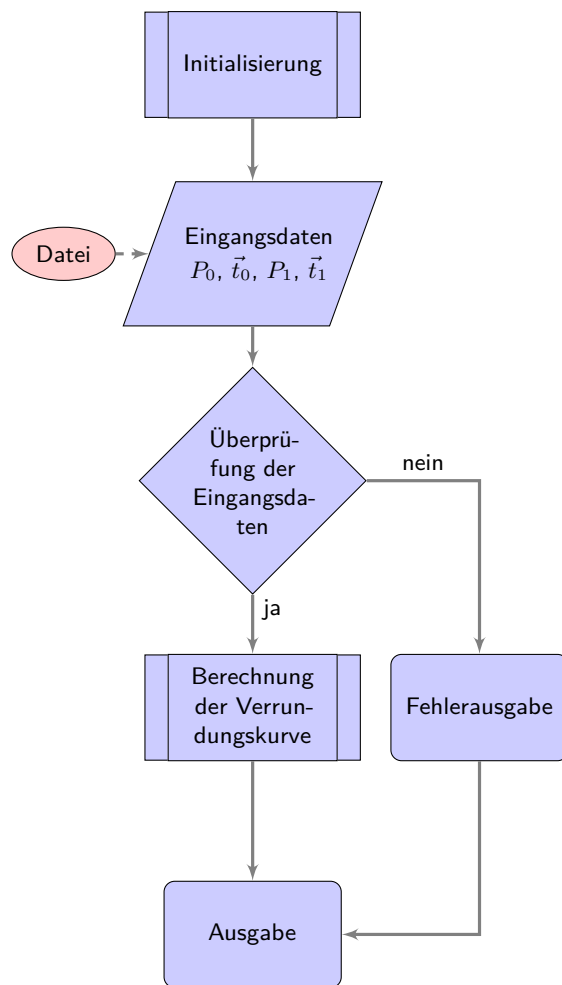


Abbildung 23.1.: Programmablaufplan „Eckenverrundung“

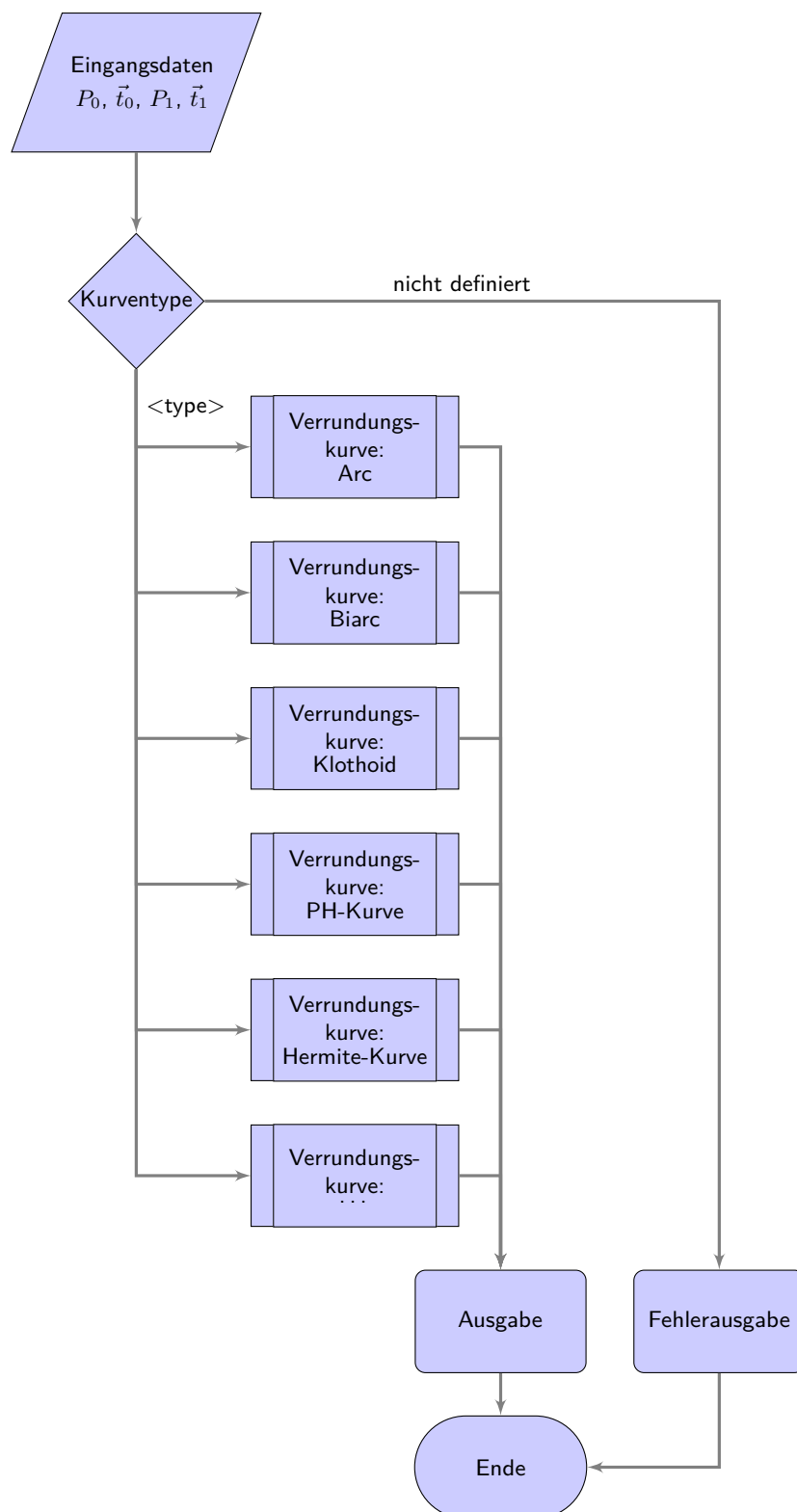


Abbildung 23.2.: Programmablaufplan „Auswahl der Verrundungsstrategien“

24. Representation of Python Programs

It is possible to integrate a part into the document, see 24.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document. It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 24.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file `Blink.py`.

```

1000 %%
1001 %% This is file 'rename-to-empty-base.tex',
1002 %% generated with the docstrip utility.
1003 %%
1004 %% The original source files were:
1005 %%
1006 %% fileerr.dtx (with options: 'return')
1007 %%
1008 %% This is a generated file.
1009 %%
1010 %% The source is maintained by the LaTeX Project team and bug
1011 %% reports for it can be opened at https://latex-project.org/bugs/
1012 %% (but please observe conditions on bug reports sent to that address!)
1013 %%
1014 %%
1015 %% Copyright (C) 1993–2025
1016 %% The LaTeX Project and any individual authors listed elsewhere
1017 %% in this file.
1018 %%
1019 %% This file was generated from file(s) of the Standard LaTeX ‘Tools
1020 %% Bundle’.
1021 %%
1022 %%
1023 %% It may be distributed and/or modified under the
1024 %% conditions of the LaTeX Project Public License, either version 1.3c
1025 %% of this license or (at your option) any later version.
1026 %% The latest version of this license is in
1027 %% https://www.latex-project.org/lppl.txt
1028 %% and version 1.3c or later is part of all distributions of LaTeX
1029 %% version 2005/12/01 or later.
1030 %%
1031 %% This file may only be distributed together with a copy of the LaTeX
1032 %% ‘Tools Bundle’. You may however distribute the LaTeX ‘Tools Bundle’
1033 %% without such generated files.
1034 %%
1035 %% The list of all files belonging to the LaTeX ‘Tools Bundle’ is
1036 %% given in the file ‘manifest.txt’.
1037 %%
1038 \message{File ignored}
1039 \endinput
1040 %%
1041 %% End of file 'rename-to-empty-base.tex'.

```

../Code/HelloWorld/Blink.py

25. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 25.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 25.1.: The program “Hello World” for an Arduino microcontroller board is inserted from the file `Blink.ino`.

```

1000 /**
1001  *
1002  *   @file Blink.ino
1003  *
1004  *   @brief Simple program for testing the configuration
1005  *
1006  *   Turns an LED on for one second, then off for one second, repeatedly.
1007  *
1008  *   Most Arduinos have an on-board LED you can control. On the UNO, MEGA
1009  *   and ZERO
1010  *   it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is
1011  *   set to
1012  *   the correct LED pin independent of which board is used.
1013  *   If you want to know what pin the on-board LED is connected to on your
1014  *   Arduino
1015  *   model, check the Technical Specs of your board at:
1016  *
1017  *   https://www.arduino.cc/en/Main/Products
1018  *
1019  *   modified 8 May 2014
1020  *   by Scott Fitzgerald
1021  *   modified 2 Sep 2016
1022  *   by Arturo Guadalupi
1023  *   modified 8 Sep 2016
1024  *   by Colby Newman
1025  *
1026  *   This example code is in the public domain.
1027  *
1028  *   https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
1029  */
1030
1031
1032 /**
1033  *   @brief the setup function runs once when you press reset or power the
1034  *   board
1035  *
1036  *   standard function of Arduino sketches
1037  *
1038  *   Initialization of the pin LED_BUILTIN as output
1039  *
1040  *   @param —
1041  *
1042  *   @return void
1043  */
1044 void setup() {
1045     // initialize digital pin LED_BUILTIN as an output.
1046     pinMode(LED_BUILTIN, OUTPUT);
1047 }
1048
1049 /**
1050  *   @brief the loop function runs over and over again forever
1051  *
1052  *   standard function of Arduino sketches
1053  *
1054  *   swichting the led on / off for 1sec.
1055  *
1056  *   @param —
1057  *
1058  *   @return void
1059  */
1060 void loop() {
1061     digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the
1062     voltage level)
1063     delay(1000); // wait for a second
1064     digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the
1065     voltage LOW
1066     delay(1000); // wait for a second
1067 }

```

../Code/Arduino/Blink/Blink.ino

26. Erstes Kapitel

...

Eine Speicherprogrammierbare Steuerung (SPS) ist ...

Eine Computerized Numerical Control (CNC) benötigt eine SPS (SPS) zur ...

27. CAGD

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Das Buch CAGD von Gerald Farin ist ein Klassiker über Splines. [Far02]

Die Norm 66025 zur Programmierung von CNC-Maschinen ist ebenfalls ein Klassiker; sie behandelt allerdings keine Splines. [DIN66025-2]

Herr F. Farouki hat sich sowohl mit der Programmierung von CNC-Maschinen als auch mit Splines beschäftigt. Sein Artikel¹ über einen Echtzeitinterpolator zeigt dies auch. [FS17]

Ein neue Dimension der Werkzeugmaschinen sind durch die Erfindung von 3D-Drucker entstanden. [Rus+07]

Ein anderer Aspekt der Automatisierungstechnik ist die Kommunikation. Durch die 5G-Technik ist hier ein weiterer Meilenstein erreicht worden.²

¹Mitautor ist J. Srinathu

²Zaf+20.

A. Zeichnungen mit tikz

Das Paket tikz ist ein mächtiges Werkzeug zu erstellen von Grafiken. Es existieren vielen Einführungen. Hier werden nur die ersten Schritte gezeigt, so dass man leicht Flussdiagramme erzeugen kann.

Zeichnen einer Linie und Pfeile

B. Kriterien für eine gutes L^AT_EX-Projekt

Ein guter Bericht zeichnet sich nicht nur durch den guten Inhalt aus, sondern erfüllt auch formale Aspekte. Die folgende Liste soll helfen, grundlegende Regeln einzuhalten. Vor der Abgabe sollten alle Punkte geprüft und abgehakt werden.

- ☐ Wird eine geeignete Verzeichnisstruktur verwendet?
- ☐ Wird eine geeignete Aufteilung in Dateien verwendet?
- ☐ Werden aussagekräftige Verzeichnis- und Dateinamen verwendet?
 - ☐ Werden Verzeichnis- und Dateinamen wie z.B. report oder Hausarbeit vermieden?
 - ☐ Werden für Bilder aussagekräftige Namen und nicht „image1“ verwendet?
 - ☐ Sind die Verzeichnis- und Dateinamen nicht zu lang?
 - ☐ Werden Sonderzeichen und Freizeichen vermieden?
- ☐ Werden Befehle wie `\newline` und `\\` vermieden?
- ☐ Werden die L^AT_EX-Dateien übersichtlich gestaltet?
 - ☐ Werden in den L^AT_EX-Dateien Einrückungen verwendet?
 - ☐ Werden Freizeilen eingefügt?
 - ☐ Wird im Header der Dateien das Projekt erwähnt?
 - ☐ Wird im Header der Dateien die Hauptquellen erwähnt?
 - ☐ Wird im Header der Dateien der Autor erwähnt?
- ☐ Werden alle notwendigen Dateien abgegeben?
- ☐ Werden die temporären Dateien gelöscht?
- ☐ Werden Grafiken selber erstellt?
- ☐ Werden die Quellen der Bilder angegeben?
- ☐ Wird mit dem Kommando `\cite` zitiert?
- ☐ Wird eine bib-Datei verwendet?
- ☐ Sind die Einträge der bib-Datei übersichtlich?
- ☐ Sind die Einträge der bib-Datei so editiert, dass sie korrekt dargestellt werden?
- ☐ Werden die korrekten Typen für die bib-Einträge verwendet?
- ☐ Werden aussagekräftige Schlüssel in den bib-Dateien verwendet?

Leider kann die Liste nicht vollständig sein, aber sie liefert einige Anhaltspunkte.

Index

- 3D-Drucker, 129
- 5G, 129
- Speicherprogrammierbare Steuerung, 127
 - Datei
 - Blink.ino, 126
 - Blink.py, 124
 - PackageExample.py, 91
 - requirements.txt, 89
- Abschluss und Fazit der Applikation, 87
- Applikation auf dem Arduino, 87
- Arduino Mbed OS, 87
- Arduino Nano 33 BLE Sense, 87
 - Arduino IDE Konfiguration, 87
 - Bibliotheken Arduino Nano 33 BLE Sense, 87
 - BLE Sense Board, 87
 - Board Manager Eintrag, 87
 - Nano 33 BLE, 87
 - Stromversorgung Nano 33 BLE Sense, 87
- Barometer, 87
- Bluetooth Low Energy, 87
- CAGD, 129
 - Splines, 129
- CNN, 11, 127
- Computerized Numerical Control
 - siehe* CNN, 11, 127
- Datenverarbeitung auf dem Arduino, 87
- Distanzmessung mit Servo, 87
- Domain Knowledge Application, 87
- eingebaute Sensoren, 87
- Example, 91
 - Description, 91
 - Installation, 91
 - Introduction, 91
- HC-SR04, 87
- IMU Sensor, 87
- Interrupts und Timer, 87
- Messzyklus Arduino, 87
- Monitoring der Messdaten, 87
- nRF52840 Mikrocontroller, 87
- Onboard Mikrofon, 87
- Radar-Applikation, 87
- scannender Ultraschall-Radar, 87
- Serial Plotter Ausgabe, 87
- Servomotor
 - Drehbereich SG90, 87
 - Getriebe Servo, 87
 - mechanische Befestigung Servo, 87
 - Positionsregelung Servo, 87
 - PWM-Signal Servo, 87
 - Servostromaufnahme, 87
 - Servohorn, 87
 - Stellwinkel Servo, 87
- Servomotor TowerPro SG90, 87
- SG90 Servomotor, 87
- Softwarearchitektur Radar, 87
- Speicherprogrammierbare Steuerung
 - siehe* SPS, 11, 127
- SPS, 11, 127, *siehe* Speicherprogrammierbare Steuerung
- State Machine Applikation, 87
- Temperatur- und Feuchtigkeitssensor, 87
- TowerPro SG90, 87
- Ultraschallsensor, 87
 - Ansteuerung HC-SR04 mit Arduino, 87
 - Echo Pin, 87
 - Entfernungsmessung Ultraschall, 87
 - Messbereich HC-SR04, 87
 - Messfehler Ultraschallsensor, 87
 - Montageposition Ultraschallsensor, 87
 - Störquellen Ultraschall, 87
 - Trigger Pin, 87
 - Versorgungsspannung HC-SR04, 87
 - Zeit-zu-Distanz-Berechnung, 87
- Ultraschallsensor HC-SR04, 87
- Verification und Evaluation, 87
- Visualisierung Radardaten, 87